



北京航空航天大学
BEIHANG UNIVERSITY



数据结构与程序设计（信息类）

Data Structure & Programming

北京航空航天大学 数据结构课程组

软件学院 谭火彬

2024年春



提纲：线性表

- ◆ 2.1 线性表的基本概念
- ◆ 2.2 线性表的顺序存储
- ◆ 2.3 线性表的链式存储：线性链表
- ◆ 2.4 循环链表
- ◆ 2.5 双向链表



提纲：线性表

- ◆ 2.1 线性表的基本概念
- ◆ 2.2 线性表的顺序存储
- ◆ 2.3 线性表的链式存储：线性链表
- ◆ 2.4 循环链表
- ◆ 2.5 双向链表



2.1 线性表的基本概念

◆线性表: $A=(a_1, a_2, \dots, a_n)$

◆线性关系

- ✓(1) 当 $1 < i < n$ 时, a_i 的直接前驱为 a_{i-1} , a_i 的直接后继为 a_{i+1}
- ✓(2) 除了第一个元素与最后一个元素, 序列中任何一个元素有且仅有一个直接前驱元素, 有且仅有一个直接后继元素
- ✓(3) 数据元素之间的先后顺序为 “**一对一**” 的关系

◆线性表

- ✓数据元素之间逻辑关系为线性关系的数据元素集合称为线性表
- ✓数据元素的个数 n 为线性表的长度, 长度为 0 的线性表称为空表

◆线性表的特点: (1) 同一性 (2) 有穷性 (3) 有序性

	学 号	姓 名	性 别	年 龄	其 他
a_1	99001	张 华	女	17	
a_2	99002	李 军	男	18	
a_3	99003	王 明	男	17	
$\vdots$	...	...	...	...	
$\vdots$	...	...	...	...	
$\vdots$	...	...	...	...	
a_{50}	99050	刘 东	女	19	



示例：线性表

$$A=(a_1, a_2, a_3, \dots, a_n)$$

数据元素为
单个整数

① 数列： (25, 12, 78, 34, 100, 88)
 $a_1 \quad a_2 \quad a_3 \quad a_4 \quad a_5 \quad a_6$

② 字母表： ('A', 'B', 'C', \dots, 'Z')
 $a_1 \quad a_2 \quad a_3 \quad \dots \quad a_{26}$

数据元素为
单个字母

③ 数据表（文件）：

	学号	姓名	性别	年龄	其他
a_1	99001	张 华	女	17	
a_2	99002	李 军	男	18	
a_3	99003	王 明	男	17	
	...	...	...	...	
⋮	...	...	...	...	
⋮	...	...	...	...	
a_{50}	99050	刘 东	女	19	

数据元素
为复杂数据



线性表的基本操作

1. **创建**一个新的线性表。
2. 求线性表的长度。
3. **检索**线性表中第 i 个数据元素($1 \leq i \leq n$)。
4. 根据数据元素的某数据项(通常称为关键字)的值求该数据元素在线性表中的位置 (**查找**) 。
5. 在线性表的第 i 个位置上存入一个新的数据元素。
6. 在线性表的第 i 个位置上**插入**一个新的数据元素。
7. **删除**线性表中第 i 个数据元素。
8. 对线性表中的数据元素按照某一个数据项的值的大小做升序或者降序**排序**。
9. **销毁**一个线性表。
10. **复制**一个线性表。
11. 按照一定的原则, 将两个或两个以上的线性表**合并**成为一个线性表。
12. 按照一定的原则, 将一个线性表**分解**为两个或两个以上的线性表。

.....



线性表的基本操作（函数原型）

```
createList(NodeType *list, int length);    //创建一个空表
destroyList (NodeType *list, int length);  //销毁一个表
printList(NodeType *list, int length);    //输出一个表
getNode (NodeType *list, int pos);        //获取表中指定位置元素
searchNode(NodeType *list, Type node);    //在表中查找某一元素
//在表中指定位置插入一个结点
insertNode(NodeType *list, int pos, NodeType node);
deleteNode(NodeType *list, int pos);      //在表中指定位置删除一个结点
.....
```



提纲：线性表

- ◆ 2.1 线性表的基本概念
- ◆ 2.2 线性表的顺序存储
 - ✓ 2.2.1 基本原理
 - ✓ 2.2.2 主要操作：（折半）查找、插入、删除
 - ✓ 2.2.2 综合应用
- ◆ 2.3 线性表的链式存储：线性链表
- ◆ 2.4 循环链表
- ◆ 2.5 双向链表



2.2.1 线性表的顺序存储

◆构造原理

- ✓用一组地址连续的存储单元依次存储线性表的数据元素，数据元素之间的逻辑关系通过数据元素的存储位置直接反映

($a_1, a_2, a_3, \dots, a_n$)

k个单元



LOC(a_i)

所谓一个元素的地址是指该元素占用的k个(连续的)存储单元的第一个单元的地址



顺序存储结构

结论

若假设每个数据元素占用 k 个存储单元，并且已知第一个元素的存储位置 $LOC(a_1)$ ，则有

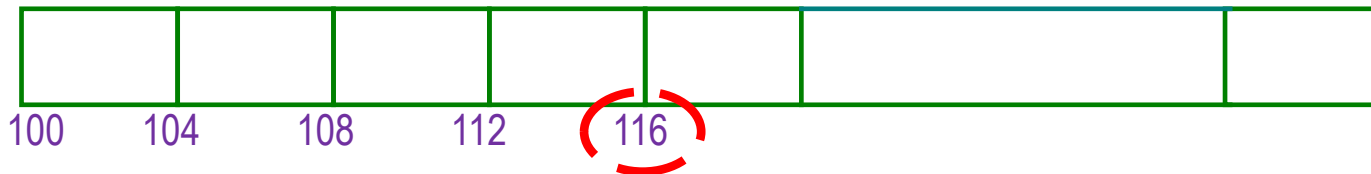
$$LOC(a_i) = LOC(a_1) + (i-1) \times k$$

提问

这个如此简单的公式，说明了顺序存储的线性表具有一个什么样的巨大优势呢？

例：

$$LOC(a_1)=100 \quad k=4 \quad \text{求} LOC(a_5)=?$$

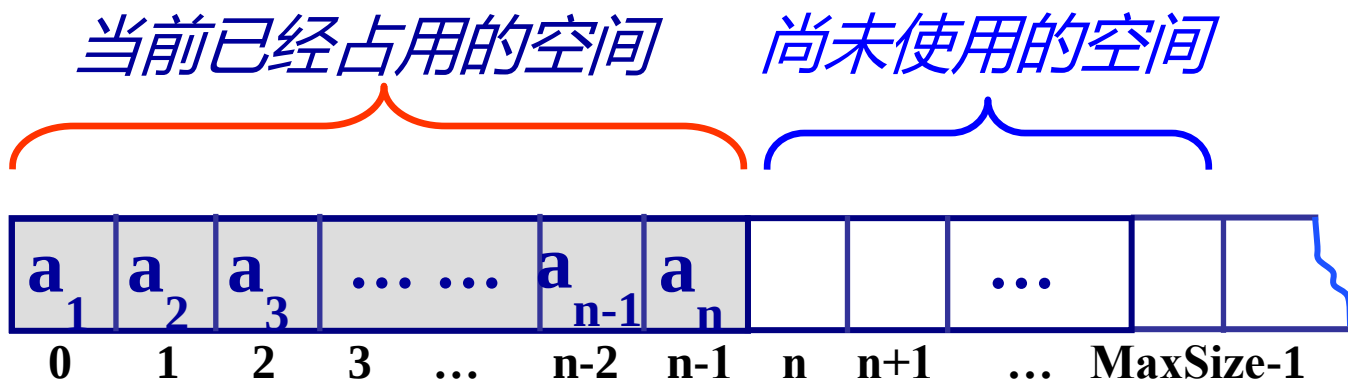


$$LOC(a_5) = 100 + (5-1) \times 4 = 116$$



顺序存储结构示意图

$(a_1, a_2, a_3, \dots, a_{n-1}, a_n)$



事先分配给线性表的空间

$n < \text{MaxSize}$



C语言实现顺序存储结构：数组

$$A = (a_1, a_2, a_3, \dots, a_{n-1}, a_n)$$

```
#define MaxSize 100
typedef int ElemType;
ElemType list[MaxSize];
int N;
```

预先分配给线性表的空间大小

表的长度



2.2.2 主要操作：查找

1. 查找：确定元素item在长度为N的顺序表list中的位置

$(a_1, a_2, a_3, \dots, a_{i-1}, a_i, a_{i+1}, \dots, a_{n-1}, a_n)$

... ↑

顺序查找法

```
int searchElem(ElemType list[], int N, ElemType item)
{
    int i;
    for (i = 0; i < N; i++)
        if (list[i] == item)
            return i; // 查找成功，返回在表中位置
    return -1;        // 查找失败，返回信息-1
}
```

时间复杂度 $O(n)$

注意逻辑位置和物理位置的区别：
C语言中第一个元素的下标是0



更高效的查找算法：折半查找

- ◆ 假设数据集按由小到大排列，**折半查找**算法的核心思想是：
 - ✓ 将要查找的有序数据集的**中间元素**与指定数据项相比较
 - ✓ 如果指定数据项小于该中间元素，则将数据集的前半部分指定为要查找的数据集，然后转步骤1
 - ✓ 如果指定数据项大于该中间元素，则将数据集的后半部分指定为要查找的数据集，然后转步骤1
 - ✓ 如果指定数据项等于中间元素，则查找成功结束
 - ✓ 最后如果数据集中没有元素再可进行查找，则查找失败



示例：折半查找过程

例：在下面有序数据集中查找数据项62

0	1	2	3	4	5	6	7	8	9
5	7	16	24	25	50	45	50	62	65

↑ ← 查找范围 → ↑
low high

$item > data[mid]$, 即 $62 > 25$

0	1	2	3	4	5	6	7	8	9
5	7	16	24	25	50	45	50	62	65

↑ ← 查找范围 → ↑
low high

$item > data[mid]$, 即 $62 > 50$

0	1	2	3	4	5	6	7	8	9
5	7	16	24	25	50	45	50	62	65

↑ ← 查找范围 → ↑
low high

$item = data[mid]$, 即 $62 = 62$

item = 62(查找顶)

low = 0(查找范围开始)

high = 9(查找范围结束)

mid = $(low+high)/2=4$ (查找范围中间)

low = mid+1=5

high = 9

mid = $(low+high)/2=7$

low = mid+1=8

high = 9

mid = $(low+high)/2=8$



折半查找算法的实现

//在有序（递增）顺序表list中查找给定元素的折半查找算法如下：

```
int searchElem(ElemType list[], int N, ElemType item)
{
    int low = 0, high = N - 1, mid;
    while (low <= high)
    {
        mid = (high + low) / 2;
        if (item < list[mid])
            high = mid - 1;
        else if (item > list[mid])
            low = mid + 1;
        else
            return (mid);
    }
    return -1;
}
```

时间复杂度
 $O(\log_2 n)$

也就是说假如一个顺序表有1024个元素，顺序查找算法平均查找次数为512次，而折半查找算法最多只须10次



主要操作：插入

2.插入：在长度为n的顺序表list的第i个位置上插入一个新的数据元素item

在线性表的第i-1个数据元素与第i个数据元素之间插入一个由符号item表示的数据元素，使得长度为n的线性表

$$(a_1, a_2, \dots, a_{i-1}, a_i, a_{i+1}, \dots, a_{n-1}, a_n)$$

n个数据元素

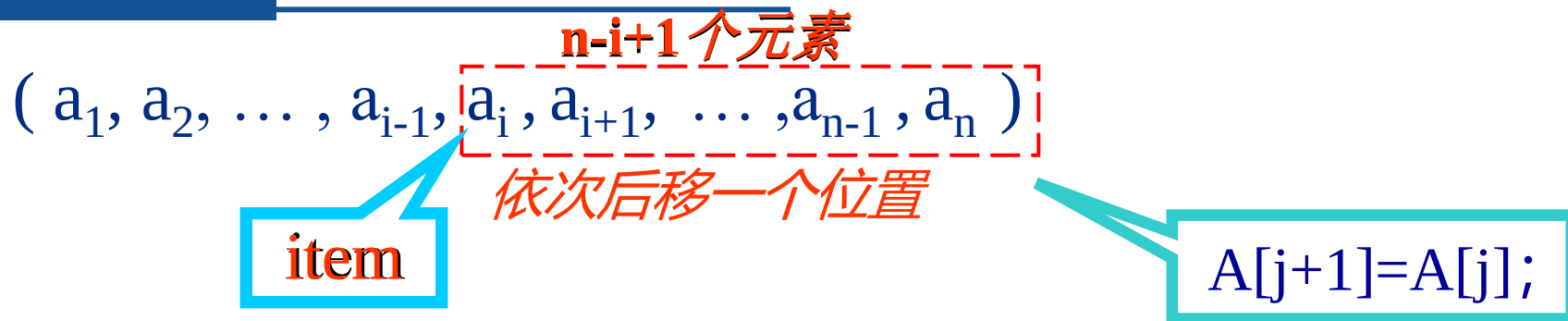
转换成长度为n+1的线性表

$$(a_1, a_2, \dots, a_{i-1}, \text{item}, a_i, \dots, a_{n-1}, a_n)$$

n+1个数据元素



插入算法：过程解析



正常情况下需要做的工作：

- (1) 将第 i 个元素至第 n 个元素依次后移一个位置；
- (2) 将被插入元素插入表的第 i 个位置；
- (3) 修改表的长度(表长增1)。($n++;$)

插入操作需要考虑的异常情况：

- (1) 是否表满？($n==\text{MaxSize}$)
- (2) 插入位置是否合适？(正常位置: $0 \leq i \leq n$)



插入算法：实现

```
//假设N是顺序表的长度（元素个数），为一个全局变量
int insertElem(ElemType list[], int i, ElemType item)
{
    int k;
    //空间已满，或插入位置不对
    if (N == MaxSize || i < 0 || i > N)
        return -1; // 插入失败，约定返回-1

    for (k = N - 1; k >= i; k--)
        list[k + 1] = list[k]; // 依次后移原来的元素
    list[i] = item;           // 将item插入到第i个位置
    N++;                     // 链表长度+1
    return 1;
}
```

该算法的时间复杂度是： **$O(n)$**



算法分析：元素的移动次数的平均值

◆元素移动次数的平均值：衡量插入和删除算法时间效率的另一个重要指标

✓若设 p_i 为插入一个元素于线性表第 i 个位置的概率(概率相等)，则在长度为 n 的线性表中插入一个元素需要移动其他的元素的平均次数为

$$T_{is} = \sum p_i(n-i+1) = \sum (n-i+1)/(n+1) = n/2$$



主要操作：删除

3.删除：删除长度为n的顺序表list的某个数据元素

把线性表的第i个数据元素从线性表中去掉，使得长度为n 的线性表

$(a_1, a_2, \dots, a_{i-1}, a_i, a_{i+1}, \dots, a_{n-1}, a_n)$

n个数据元素

转换成长度为 n-1 的线性表

$(a_1, a_2, \dots, a_{i-1}, a_{i+1}, \dots, a_{n-1}, a_n)$

n-1个数据元素



删除算法：过程解析

$(a_1, a_2, \dots, a_{i-1}, a_i, \underbrace{a_{i+1}, \dots, a_{n-1}, a_n}_{\text{依次前移一个位置}})$

$\overbrace{a_{i+1}, \dots, a_{n-1}, a_n}^{n-i \text{ 个元素}}$

`list[j-1]=list[j];`

正常情况下需要做的工作：

- (1) 将删除元素的下一元素至第n个元素依次前移一个位置；
- (2) 修改表的长度(表长减1)。 (`n--;`)

删除操作需要考虑的异常情况：

- (1) 是否表空？ $N=0$?
- (2) 删除位置是否合适？ (正常位置: $0 \leq i \leq N-1$)



删除算法：实现

```
// 假设N是表的长度（元素个数），为一个全局变量
int deleteElem(ElemType list[], int i)
{
    int k;
    // 测试表空，或者删除的位置不对
    if (N == 0 || i < 0 || i > N - 1) return -1;
    for (k = i + 1; k < N; k++)
        list[k - 1] = list[k]; // 元素依次往前移一个位置
    N--;                       // 线性表的长度减1
    return 1;
}
```

该算法的时间复杂度是： $O(n)$

元素的平均移动次数： $T_{ds} = \sum p_i(n-i) = \sum (n-i)/n = (n-1)/2$



顺序存储结构的特点

◆优点

- ✓ (1) 构造原理简单、直观，易理解
- ✓ (2) 元素存储地址可以通过下标进行访问，读取速度快
- ✓ (3) 由于只需存放数据元素本身的信息，而无其他空间开销，相对链式存储结构而言，存储空间开销小
- ✓ (4) 对于有序表，可使用折半查找等快速查找算法，查找效率高

◆缺点

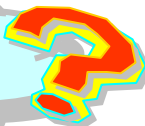
- ✓ (1) 需要提前分配足够大的地址连续的存储空间
- ✓ (2) 插入、删除等基本操作的时间效率较低，需要频繁移动元素



2.2.3 应用D02-01：有序表的插入

已知长度为 n 的非空线性表 $list$ 采用顺序存储结构,并且数据元素按值的大小非递减排列(**有序**), 写一算法, 在该线性表中插入一个数据元素 $item$, 使得线性表仍然保持按值非递减排列。

如何找到插入位置



$a_1, a_2, a_3, \dots, \overset{item}{a_i}, a_{i+1}, a_{i+2}, \dots, a_n$
↑ 依次后移一个位置

$$a_i \leq a_{i+1} \quad 1 \leq i \leq n-1$$



过程分析

需要做的工作

1. 寻找插入位置:

从表的第一个元素开始进行比较, 若有关系: $item < a_i$
则找到插入位置为表的第*i*个位置

2. 将第*i*个元素至第*n*个元素依次后移一个位置;

3. 将item插入表的第*i*个位置;

4. 表的长度增1。

例

1, 3, 5, 5, 8, 10, 13, 15, 20, 25

item=12

1, 3, 5, 5, 8, 10, 12, 13, 15, 20, 25



算法实现

```
#include <stdio.h>
#define MAXSIZE 1000
typedef int ElemType;
int N = 0;
int main()
{
    int i;
    ElemType data, list[MAXSIZE];
    scanf("%d", &N);
    for (i = 0; i < N; i++)
        scanf("%d", &list[i]);
    scanf("%d", &data);
    if (insertElem(list, data) == 1)
        printf("OK\n");
    else
        printf("Fail\n");
    return 0;
}
```

```
// 假设N是表长度，是一个全局变量
int insertElem(ElemType list[], ElemType item)
{
    int i, j;

    if (N == MAXSIZE)
        return -1;
    //寻找item的合适位置
    for (i = 0; i < N && item >= list[i]; i++)
        ;
    for (j = N - 1; j >= i; j--)
        list[j + 1] = list[j];

    list[i] = item; // 将item插入表中
    N++;
    return 1;
}
```



算法改进：利用折半查找确定元素位置

```
int insertElem(ElemType list[], ElemType item){
    int i = 0, j;

    if (N == MAXSIZE)
        return -1;
    i = searchElem(list, item); //寻找item的位置

    for (j = N - 1; j >= i; j--)
        list[j + 1] = list[j];

    list[i] = item; //将item插入表中
    N++;
    return 1;
}
```

```
//利用折半查找查询插入元素的位置
int searchElem(ElemType list[],
               ElemType item){
    int low = 0, high = n - 1, mid;
    while (low <= high) {
        mid = (high + low) / 2;
        if(( item < list[mid])
            high = mid - 1;
        else if ( item > list[mid])
            low = mid + 1;
        else
            return (mid);
    }
    return low;
}
```

算法总复杂度为： $O(\log_2 n + n) = O(n)$



综合应用D02-02：词频统计 – 顺序表

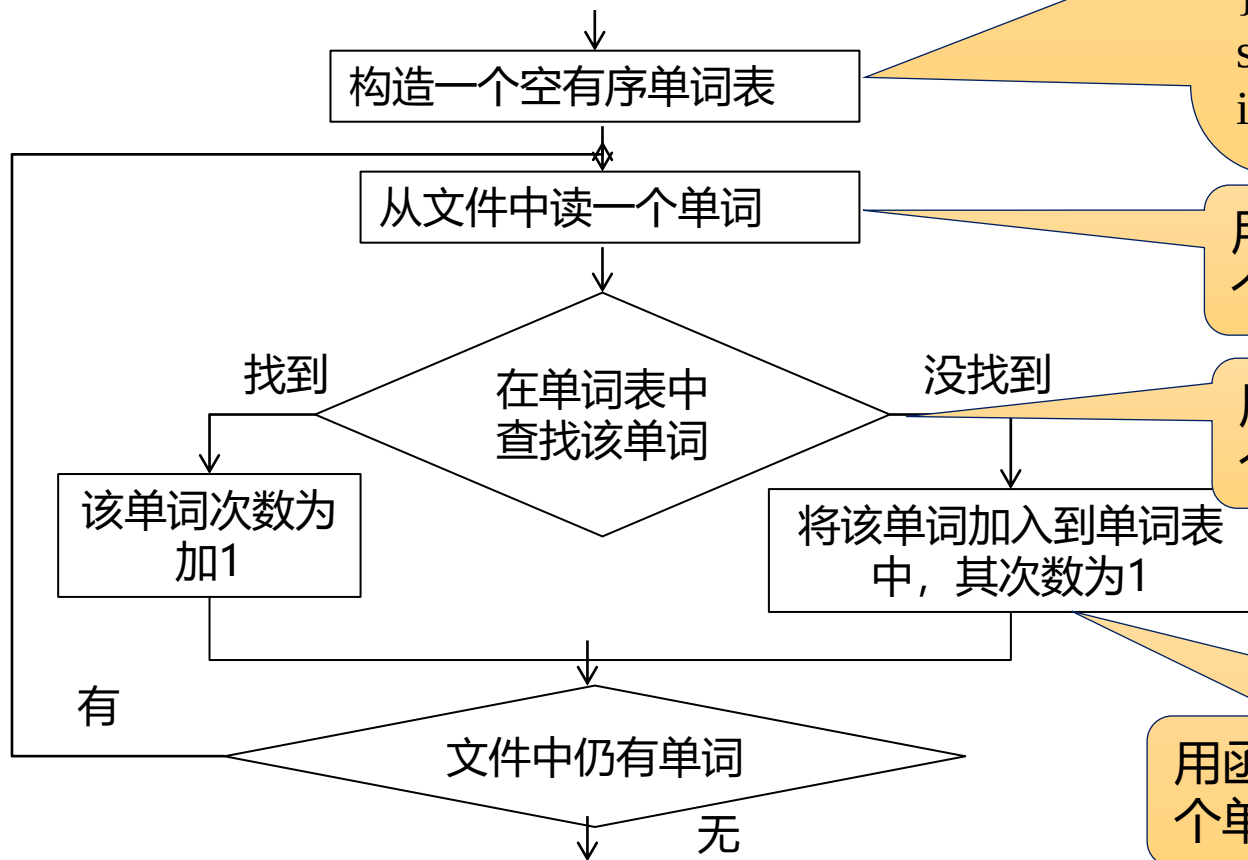
- ◆问题：编写程序统计一个文件中每个单词的出现次数（词频统计），并按字典序输出每个单词及出现次数。
- ◆算法分析：
 - ✓1.首先构造一个空的有序（字典序）单词表；
 - ✓2.每次从文件中读入一个单词；
 - ✓3.在单词表中（折半）查找该单词，若找到，则单词次数加1，否则将该单词插入到单词表中相应位置，并设置出现次数为1；
 - ✓4.重复步骤2，直到文件结束。



D02-02: 词频统计 - 顺序表

/*用数组构造一个顺序表，表中单词按字典序组织*/

```
struct lnode {  
    char word[MAXWORD];  
    int count;  
};  
struct lnode wordlist[MAXSIZE];  
int N=0; //全局变量，空表
```



用函数getWord从文件中每次读入一个单词

用函数searchWord在单词表中查找一个单词

用函数insertWord在单词表中插入一个单词



代码实现

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>
#define MAXWORD 32
#define MAXSIZE 1024
struct lnode {
    char word[MAXWORD];
    int count;
};
int getWord(FILE *bfp, char *w);
int searchWord(struct lnode list[],
               char *w);
int insertWord(struct lnode list[ ],
               int pos, char *w);
int N=0; //全局变量, 单词表实际单词个数
```

```
int main(){
    struct lnode wordlist[MAXSIZE]; /*单词表*/
    int i;
    char filename[MAXWORD], word[MAXWORD];
    FILE *bfp;

    scanf("%s", filename);
    if((bfp = fopen(bname, "r")) == NULL){
        printf("%s can't open!\n", filename);
        return -1;
    }
    while( getWord(bfp, word) != EOF)
        if(searchWord(wordlist, word) == -1) {
            printf("Wordlist is full!\n");
            return -1;
        }
    for(i=0; i<= N-1; i++)
        printf("%s %d\n", wordlist[i].word,
               wordlist[i].count);

    return 0;
}
```



代码实现：读单词

```
//从文件中读入一个单词（仅由字母组成的串），并转换成小写字母
int getWord(FILE *fp, char *w)
{
    int c;

    while(!isalpha(c=fgetc(fp)))
        if(c == EOF) return c;

    do {
        *w++ = tolower(c);
    } while(isalpha(c=fgetc(fp)));
    *w = '\0';
    return 1;
}
```




词频统计程序分析

◆由于顺序表（数组）的大小需要事先确定，用顺序表作为单词表会有什么问题？

由于事先不知道被统计的文件大小（可能是本很厚的书），单词表的大小如何定：

- 1) 太小，对于大文件会造成单词表溢出；
- 2) 太大，对一般文件处理会造成很大的空间浪费。

◆词频统计需要在单词表中频繁的查找和插入单词，有序顺序表结构的单词表有什么特点？

- 1) 单词查找效率很高（可用折半查算法，一次查找算法复杂度为 $O(\log_2 n)$ ）；
- 2) 单词表中单词需要频繁移动（对于一般的英文材料来说，插入操作较多，一次插入算法的复杂度为 $O(n)$ ）；

◆无序（输入序）顺序表结构构造的单词表又有什么特点？（考虑怎么实现？）

- 1) 单词查找效率低（顺序查找算法，一次查找算法复杂度为 $O(n)$ ）；
- 2) 单词不需要移动（新单词总是放在表尾），但需要对最终的总表要进行排序，简单排序算法的复杂度为 $O(N^2)$ ；



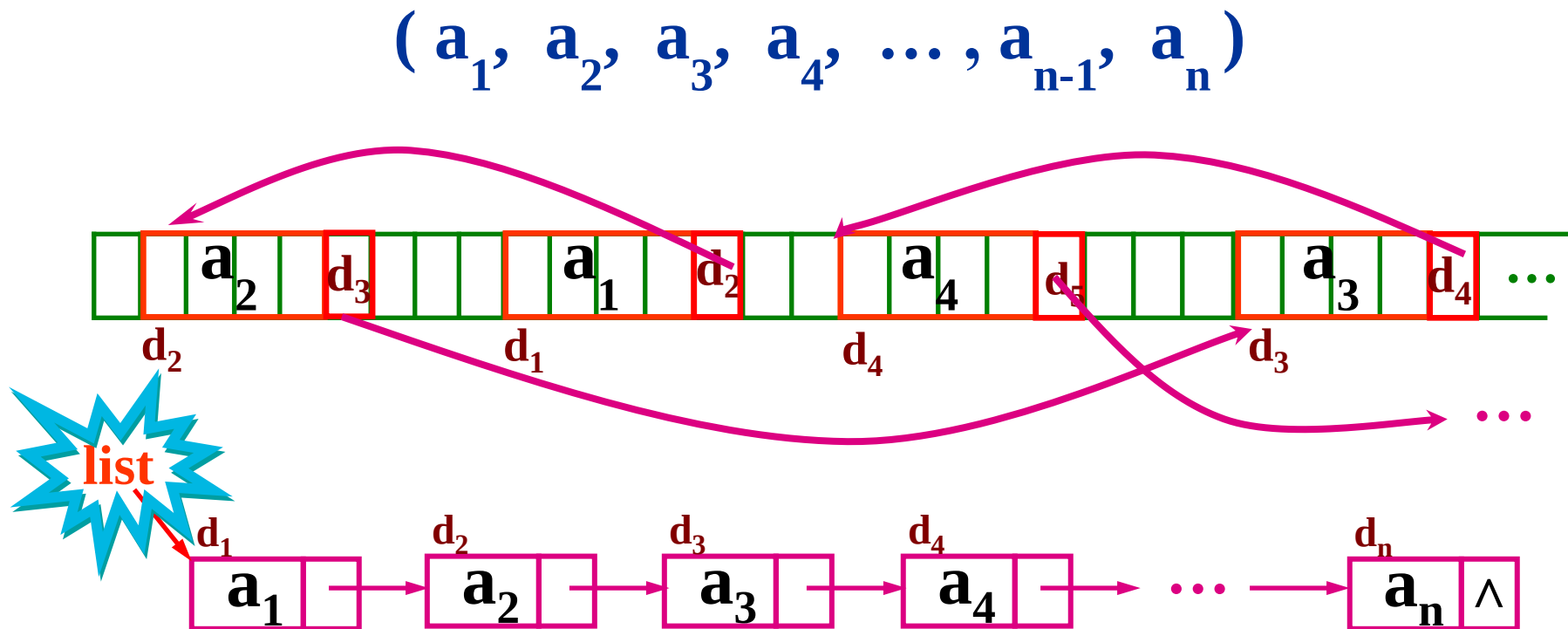
提纲：线性表

- ◆ 2.1 线性表的基本概念
- ◆ 2.2 线性表的顺序存储
- ◆ 2.3 线性表的链式存储：线性链表
 - ✓ 2.3.1 基本原理
 - ✓ 2.3.2 主要操作：创建、遍历、插入、删除
 - ✓ 2.3.3 综合应用
- ◆ 2.4 循环链表
- ◆ 2.5 双向链表



2.3 线性表的链式存储

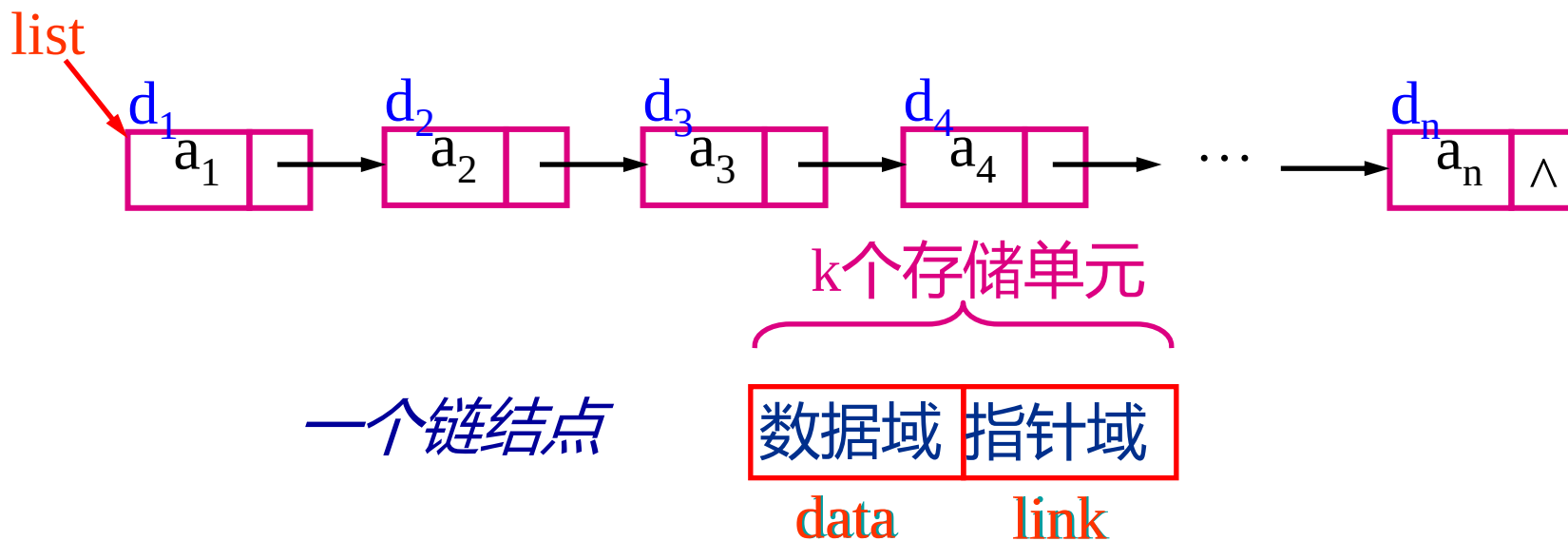
用一组地址任意的存储单元(连续的或不连续的)依次存储表中各个数据元素, 数据元素之间的逻辑关系通过**指针**间接地反映出来。





线性链表

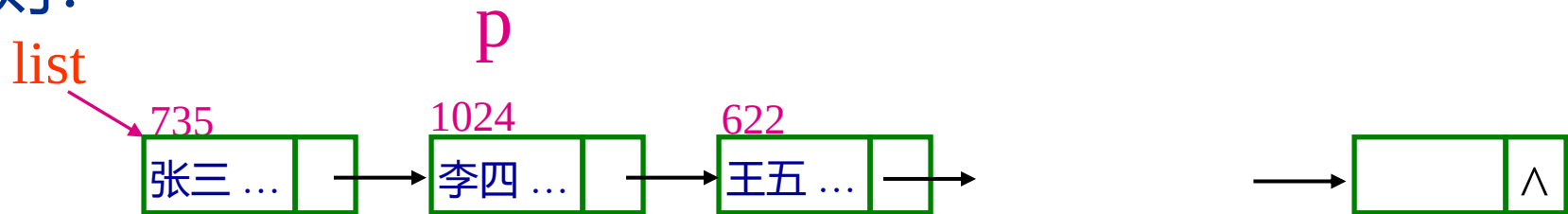
- ◆ 线性表的这种存储结构称为线性链表，又称单（向）链表
- ◆ 一般形式





线性链表 (续)

若指针变量p为指向链表中某结点的指针(即p的内容为链表中某链结点的地址)), 则:



p->data

表示由p指向的链结点的数据域

```
x = p->data;  x == "李四";
```

p->link

表示由p指向的链结点的指针域, 即p所指的链结点的下一个链结点的指针 (地址)

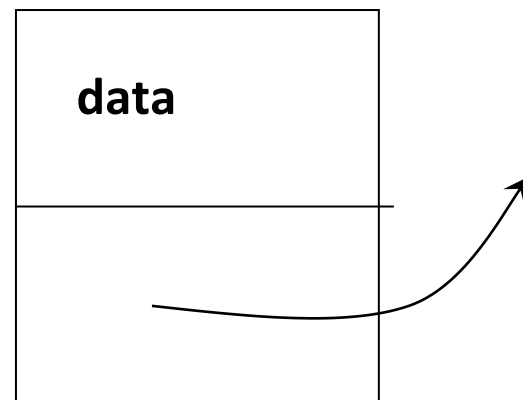


线性链表的定义：自引用结构

◆自引用结构其成员分为两部分：

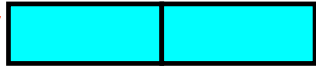
- ✓1. 数据域：存储实际的数据
- ✓2. 指针域：一个或多个指向自身结构类型的指针，用于存储数据的关系（如：下一个元素的位置）

```
struct Node
{
    ElemType data; //数据域
    struct Node *link; //指针域
};
```





自引用结构：线性链表

链结点: 
data link

```
//定义线性链表的结点
struct Node
{
    ElemType data;
    struct Node *link;
};
struct Node *list, *p;
```

使用typedef

```
//使用typedef定义线性链表结点
typedef struct _Node
{
    ElemType data;
    struct _Node *link;
}Node, *Nodeptr;

Nodeptr list, p;
```

	Num	Name	Age
a ₁	60101	张三	17
a ₂	60102	李四	16
a ₃	60103	王五	20
a ₃₀			

```
//定义数据域
typedef struct
{
    int Num;
    char Name[10];
    int Age;
} ElemType;
```

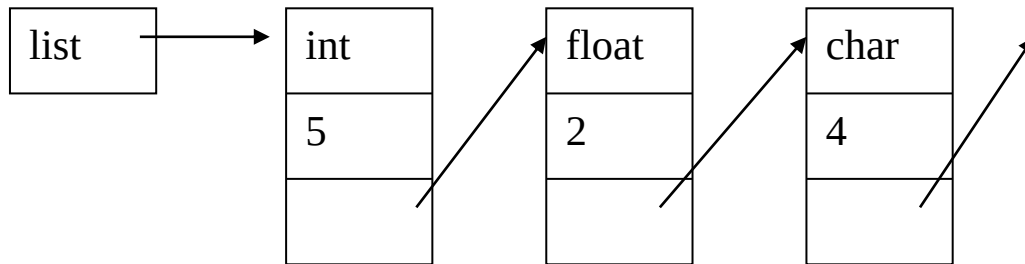
```
//使用typedef定义复杂数据结构
typedef struct _Node
{
    int Num;
    char Name[10];
    int Age;
    struct _Node *link;
}Node, *Nodeptr
```



自引用结构 (续)

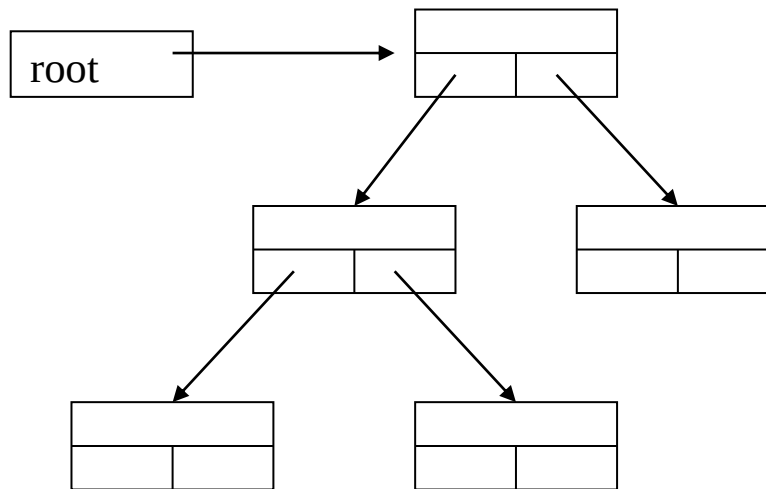
◆链表结构

```
struct word {  
    char *name;  
    int count;  
    struct word *next;  
} *list;
```



◆二叉树结构

```
struct TNode  
{  
    char *word;  
    int count;  
    struct TNode *left;  
    struct TNode *right;  
} *root;
```





2.3.2 线性链表的基本操作

- ◆ 求线性链表的长度
- ◆ **建立**一个线性链表
- ◆ 在非空线性链表的第一个结点前**插入**一个数据信息为item的新结点。
- ◆ 在线性链表中由指针q 指出的结点之后**插入**一个数据信息为item的链结点
- ◆ 在线性链表中满足某条件的结点后面插入一个数据信息为item的链结点
- ◆ 从非空线性链表中**删除**链结点q(q为指向被删除链结点的指针)
- ◆ 删除线性链表中满足某个条件的链结点
- ◆ 线性链表的逆转
- ◆ 将两个线性链表合并为一个线性链表
- ◆ 检索线性链表中的第i个链结点
- ◆



线性链表的主要操作（函数）

- `createList();` // 创建一个的链表
- `getLength(Nodeptr list);` // 获得链表的长度
- `destroyList(Nodeprt list);` // 销毁一个表
- `printList(Nodeptr list);` // 输出一个表
- `insertFirst(Nodeptr list, ElemType item);` // 在链表头插入一个元素
- `insertLast(Nodeptr list, ElemType item);` // 在链表尾插入一个元素
- // 在链表某一结点后插入包含某一个元素的结点
- `insertNode(Nodeptr list, Nodeptr p, ElemType item);`
- `searchNode(Nodeptr list, ElemType item);` // 在链表中查找某一元素
- `deleteNode(Nodeptr list, ElemType item);` // 在链表中删除包含某一元素结点



线性链表的基础操作



1) 访问第一个结点

```
p = list; //list为链表的第一个结点的地址（头指针）
```

2) 访问下一个结点

```
p = p->link; //将指针p指向下一个结点
```

3) 遍历链表（依次访问链表中的每一个结点）

```
//遍历list链表（while）  
p=list;  
while(p!=NULL)  
{  
    ...  
    p = p->link;  
}
```

```
//遍历list链表（for）  
for(p=list;p!=NULL;p=p->link)  
{  
    ...  
}
```



线性表的基础操作 (续)



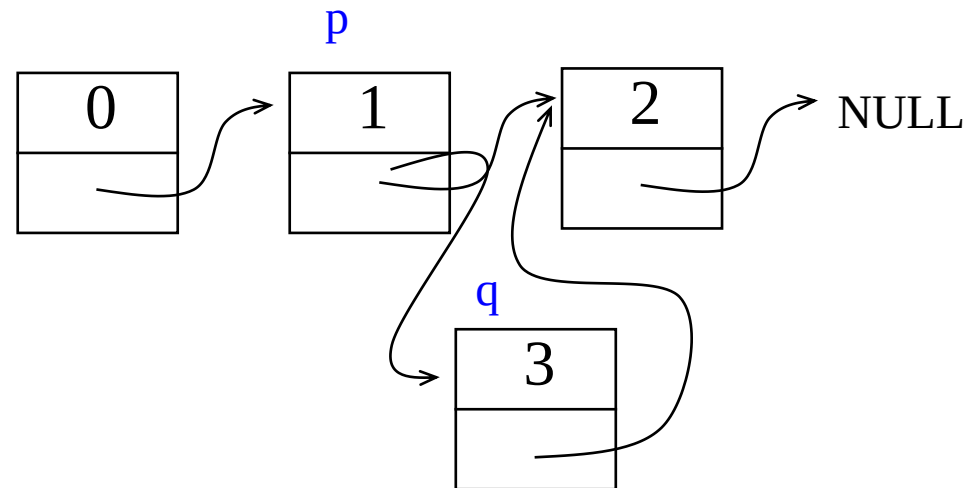
4) 创建一个结点q

```
//分配结点空间  
q = (Nodeptr)malloc(sizeof(Node));  
q->data = ... //数据  
q->link = NULL;
```



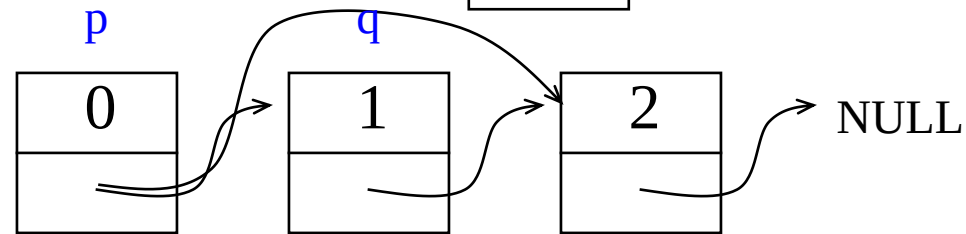
5) 当前结点p后面 插入一个结点q

```
q->link = p->link;  
p->link = q;
```



6) 删除当前结点p 后面的一个结点

```
q = p->link; //暂存待删除的结点  
p->link = p->link->link;  
//p->link = q->link;  
free(q);
```





1.创建：创建一个链表

◆操作

- ✓初始为空，依次插入数据结点，建立一个链表
- ✓返回链表头指针

◆主要要点：

- ✓创建一个结点
- ✓将创建的结点插入链表中（插入到链表的最后）
 - 链表为空时
 - 链表不为空时



建立链表：C代码实现

```
Nodeptr createList()
{
    Nodeptr list = NULL; //list是链表头指针
    Nodeptr p, q; // p指向最后一个结点, q指向新申请的结点
    int data;
    while (scanf("%d", &data) != EOF) //读入数据
    {
        q = createNode(data); // 创建一个结点
        if (list == NULL) // 链表为空
            list = p = q;
        else
            p->link = q; // 将新结点链接在链表尾部
        p = q;
    }
    return list; // 返回头指针
}
```

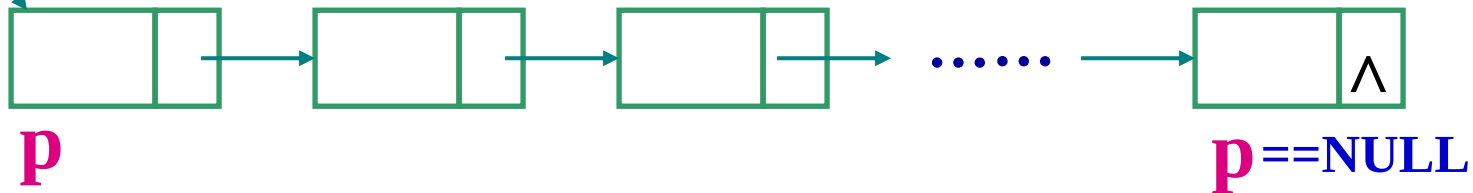
```
Nodeptr createNode(ElemType item){
    Nodeptr p;
    // 分配结点空间
    p = (Nodeptr)malloc(sizeof(Node));
    p->data = item; // 填充结点数据
    p->link = NULL; // 指针域设置为NULL
    return p;
}
```

时间复杂度 $O(n)$



2. 求长度：求链表的长度

list



```
//返回list链表的长度（结点个数）
int getLength(Nodeptr list)
{
    Nodeptr p; //临时指针，用于遍历链表
    int n = 0; //初始化链表长度为0
    //遍历链表，注意3个表达式的写法
    for (p = list; p != NULL; p = p->link)
        n++; //结点个数+1
    return n;
}
```

时间复杂度 $O(n)$



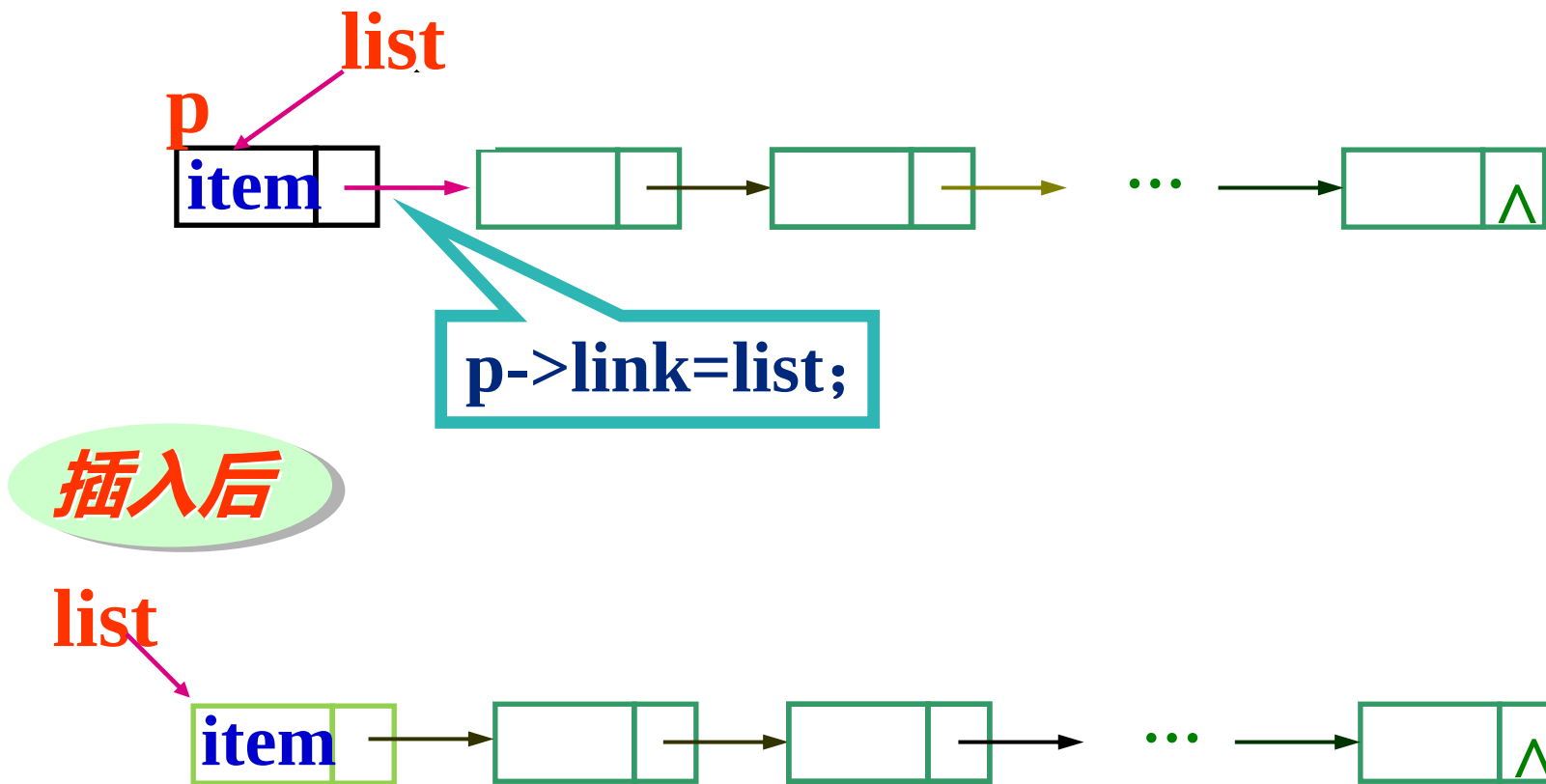
3. 查询：查询链表中的值为某个元素的结点

```
// 查询链表中值为某个元素的结点（仅返回第一个），返回结点地址
Nodeptr searchNode(Nodeptr list, ElemType item)
{
    Nodeptr p = list; //指向头指针
    while (p != NULL && p->data != item) //指针不空，且数据不想等
        p = p->link; //移动指针到下一个结点
    return p; //返回指针
}
```

时间复杂度 $O(n)$



4. 插入1：在非空线性链表第一个结点前插入数据项





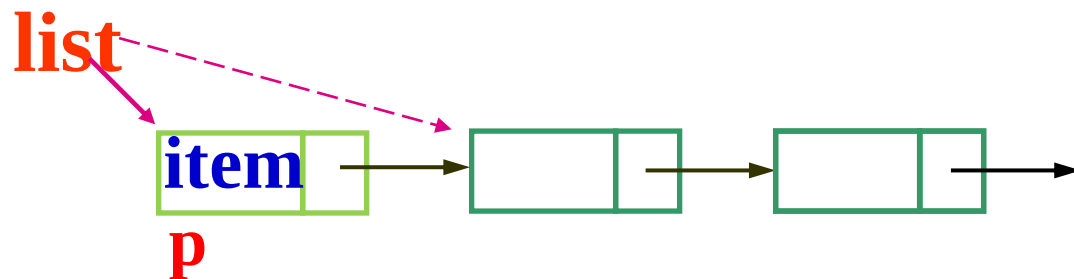
代码实现

```
// 在链表的第一个结点list前面插入一个新结点
Nodeptr insertFirst(Nodeptr list, ElemType item)
{
    Nodeptr p;
    p = createNode(item); // 创建新结点
    p->link = list;       // 将新结点的指针域指向原链表第一个结点
    return p;             // 返回新结点，作为链表新的头指针
}
```

时间复杂度 $O(1)$

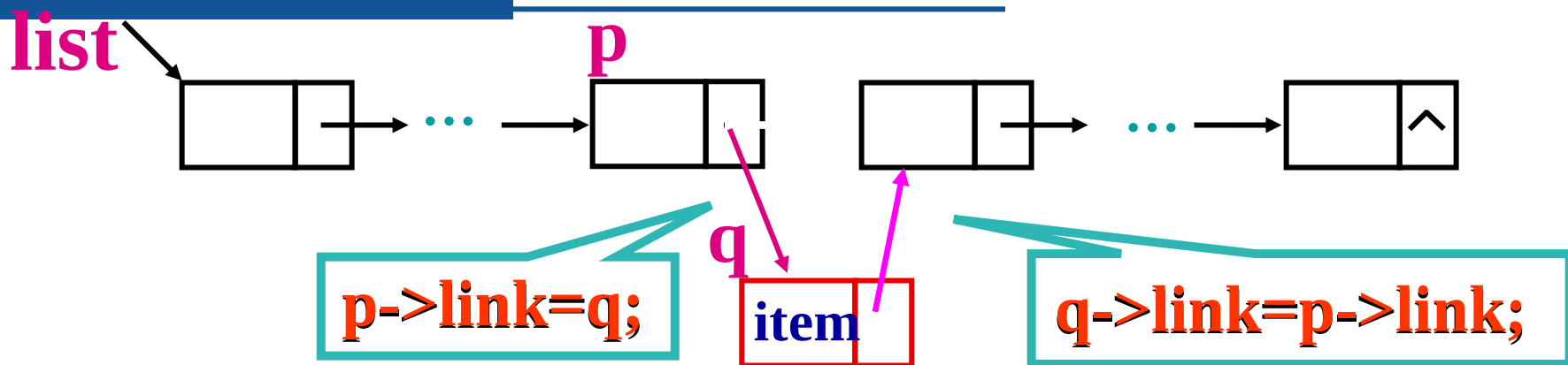
思考：为什么要返回p，
而不用：list = p?

应使用如下方式调用insertFirst函数：
list = insertFirst(list, item);





5.插入2：在链表中由指针p指的链结点之后插入

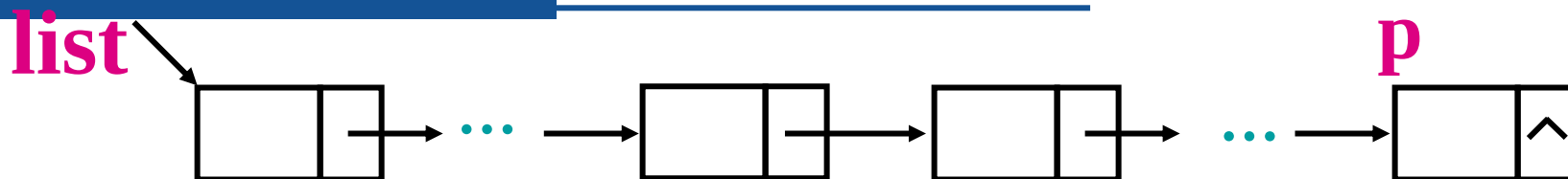


```
// 在链表中由指针p指的链结点之后插入，如果p为空，则表示插在第一个结点前面
Nodeptr insertNode(Nodeptr list, Nodeptr p, ElemType item){
    Nodeptr q;
    q = createNode(item); // 创建一个新结点
    if (p == NULL)        // 如果p为空，则表示插在第一个结点前面
        list = insertFirst(list, item);
    else
    {
        q->link = p->link; // 新结点的指针域指向p后面的结点
        p->link = q;       // p的指针域指向新结点
    }
    return list; // 返回头指针
}
```

时间复杂度O(1)



6.插入3：在链表的最后插入一个结点



```
// 在链表list的最后结点前面插入一个新结点
Nodeptr insertLast(Nodeptr list, ElemType item)
{
    Nodeptr p;
    p = createNode(item); // 创建新结点
    if (list == NULL)
        list = p; // 如果链表为空，则头指针指向p
    else
    {
        // 找到最后一个结点地址
        for (p = list; p->link != NULL; p = p->link)
            ;
        list = insertNode(list, p, item); // 在最后一个结点插入
    }
    return list; // 返回头指针
}
```

时间复杂度 $O(n)$



7.插入4：在某个特定值后面插入新结点

```
// 在链表中某个特定值（data）的后面插入新结点（item）
// 如果没找到特定值，则插在链表的最后
Nodeptr insertDataNode(Nodeptr list, ElemType data, ElemType item) {
    Nodeptr p;
    p = searchNode(list, data); // 查询某个值
    if (p != NULL)
        list = insertNode(list, p, item); // 在p后面插入新结点
    else
    {
        // 重新找到最后一个结点地址
        for (p = list; p != NULL && p->link != NULL; p = p->link)
            ;
        list = insertNode(list, p, item); // 最后一个结点后面插入
    }
    return list;
}
```

时间复杂度 $O(n)$

思考：

1. 如果链表为空，该代码是否可以处理？
2. 如果在前面插入，则需要如何处理？



8.插入5：在有序链表中相应结点后面插入

// list是一个有序升序链表，将元素item插入到相应位置上

```
Nodeptr insertSortNode(Nodeptr list, ElemType item)
```

```
{
```

```
    Nodeptr p, q; //q为插入位置前面的结点，p为插入位置后面的结点
```

```
    // 找到插入的位置
```

```
    for (p = list; p != NULL && item > p->data; q = p, p = p->link)
```

```
    ;
```

```
    if (p == list) // 在头指针前面插入
```

```
        list = insertFirst(list, item);
```

```
    else // 在结点q后插入一个结点
```

```
        list = insertNode(list, q, item);
```

```
    return list;
```

```
}
```

1. 在结点p前插入一个结点，必须要知道该结点的前序结点指针，在本程序中，q为p的前序结点指针；

2. 应使用如下方式调用insertNode函数：

```
list = insertSortNode(list, item);
```

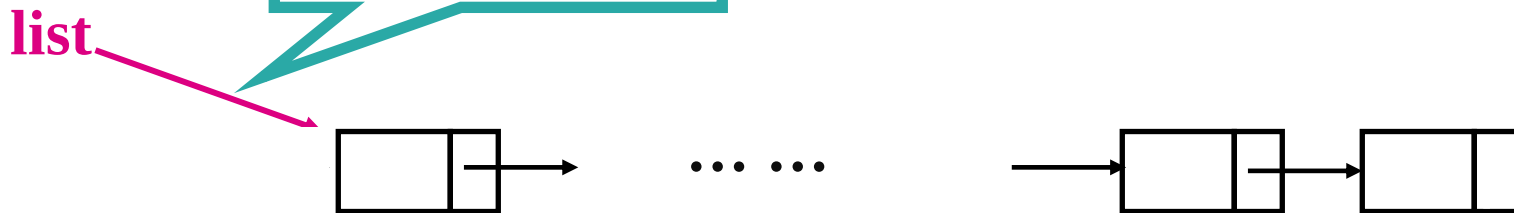
时间复杂度 $O(n)$



9.删除：删除结点p

设p的直接前驱结点由r指出

`list=p->link;`



情况1：删除链表的第一个结点



`r->link=p->link;`

情况2：删除链表中非第一个结点



9.删除1：删除某个结点后面的结点

```
// 删除链表list中r指向结点的后一个结点，list不为空，r不为空
Nodeptr deleteNextNode(Nodeptr list, Nodeptr r)
{
    Nodeptr p;
    // 判断存在需要删除的结点
    if (r != NULL && r->link != NULL)
    {
        p = r->link;           // 暂存需要删除的结点
        r->link = p->link;     // 链表中去掉删除的结点
        free(p);               // 释放删除结点的空间
    }
    return list; // 返回头指针
}
```

时间复杂度 $O(1)$



10.删除2：删除指定的结点p

```
// 删除链表list中指定的结点p
Nodeptr deleteNode(Nodeptr list, Nodeptr p) {
    Nodeptr r; // 指向p前面结点
    // 如果list或p为空，则不存在删除的结点，直接返回
    if ((list == NULL) || (p == NULL))
        return list;
    if (p == list) // 如果删除第一个结点
        list = list->link;
    else {
        // 求出p的前一个结点r
        for (r = list; r != NULL && r->link != p; r = r->link)
            ;
        if (r != NULL) // 找到前一个结点
            list = deleteNextNode(list, r);
    }
    if (p != NULL) free(p);
    return list;
}
```

时间复杂度 $O(n)$



反思：头指针的操作

- ◆链表中，使用**第一个结点的地址（头指针）**来标识（并访问）整个链表
- ◆不同于数组：数组一次性分配空间，首地址确定后**不再修改**
- ◆链表中的任何操作，都需要充分考虑涉及头指针修改的问题
 - ✓链表为空
 - ✓修改头指针：第一个结点的**地址发生变化**
 - 参考上页删除某个结点p考虑的问题
 - ✓指针作为函数参数时，无法通过函数**直接修改指针本身的值**
 - 书上使用**引用（&）参数传递**，该方法只适用于C++，纯C中无法使用
- ◆反思：能否类似于数组，将链表的头指针固定？



引入头结点

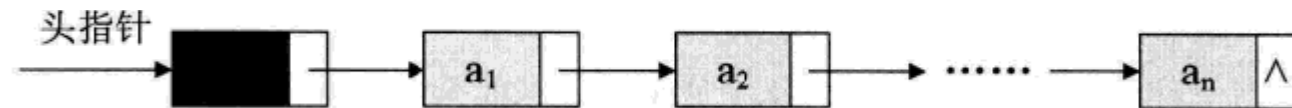
◆头结点

- ✓定义一个**空结点**（不存储任何数据），链表初始化时直接创建
- ✓该结点创建后，地址不做任何修改；链表的**头指针指向头结点**
- ✓第一个**数据结点**为头结点后面的第一个结点，后续操作只可能在头结点后面进行操作，不涉及任何头指针的修改

带头结点的空链表



带头结点的非空链表



设置头结点的最大好处是对链表结点的插入及删除操作统一了（**不用单独考虑是否需要修改头指针**）。其数据域一般无意义（也可存放链表的长度）



使用头结点

```
Nodeptr createList()
{
    Nodeptr list=NULL; // list是链表
    Nodeptr p, q; // p指向最后一个结
    int data;
    while (scanf("%d", &data) != EOF)
    {
        q = createNode(data); // 创建
        if (list == NULL) // 链表
            list = p = q;
        else
            p->link = q; // 将新结点
        p = q;
    }
    return list; // 返回头指针
}
```

//创建一个带头结点的链表

```
Nodeptr createListWithHead()
```

```
{
```

//创建头结点，数据域可不用，也可存储链表的长度（本例设为0）
//本结点不存储任何数据，任何情况下都存在，而且不会发生变化
//第一个数据存在本结点后面的一个结点
//list是链表头指针，指向头结点，后续不做任何修改

```
Nodeptr list=createNode(0);
```

Nodeptr p=list, q; // p指向最后一个结点，q指向新申请结点

```
int data;
```

```
while (scanf("%d", &data) != EOF) // 读入数据
```

```
{
```

```
q = createNode(data);
```

//不需要判断链表是否为空，必定存在头结点

```
p->link = q; // 将新结点链接在链表尾部
```

```
p = q;
```

```
}
```

```
return list; // 返回头指针
```

```
}
```

思考：带头结点后，其他操作如何修改？

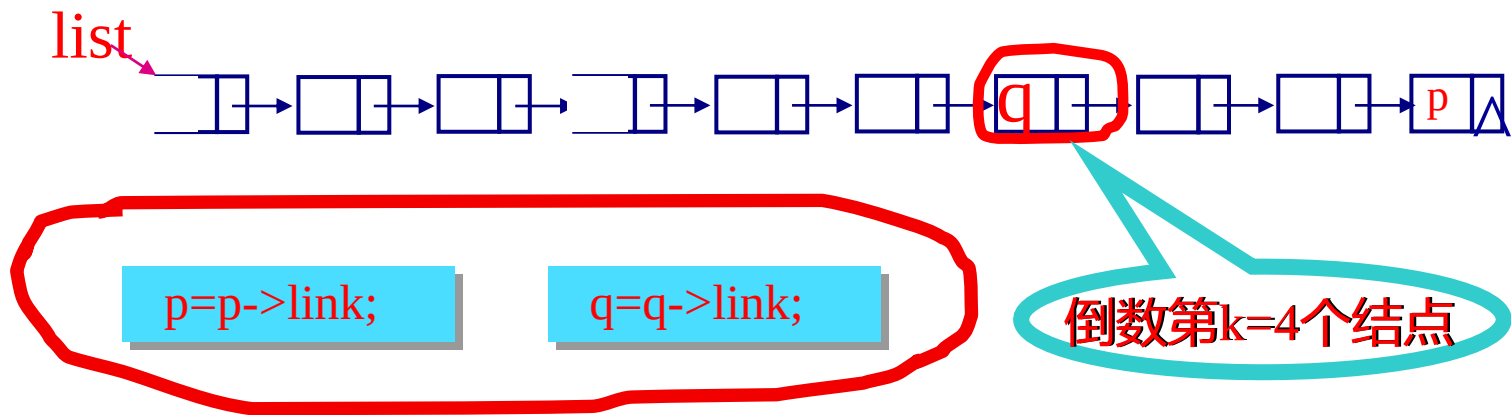


应用：D02-03-某年硕士研究生入学考试题

请写一算法，该算法用尽可能高的时间效率找到由list所指的线性链表的**倒数第k个结点**。若找到这样的结点，算法给出该结点的地址，否则，给出NULL

限制

1. 算法中不得求出链表长度;
2. 不允许使用除指针变量和控制变量以外的其他辅助空间。





实现思路

1. 设置一个指针变量 p ，初始时指向链表的第1个结点；
2. 然后令 p 后移指向链表的第 k 个结点；
3. 再设置另一个指针变量 q ，初始时指向链表的第1个结点；
4. 利用一个循环让 p 与 q 同步沿链表向后移动；当 p 指向链表最后那个结点时， q 指向链表的倒数第 k 个结点



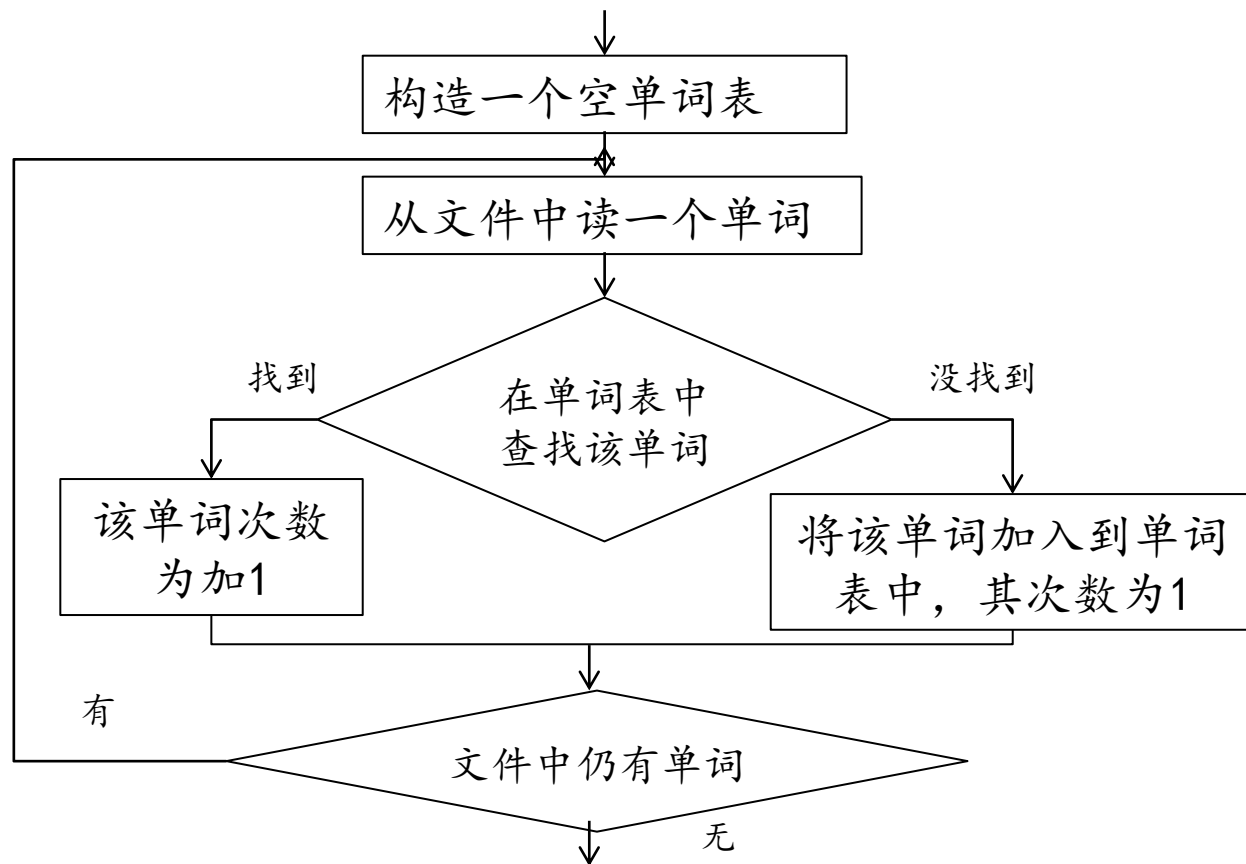
算法实现

```
//查询链表中的倒数第k个结点
Nodeptr searchLastKNode(Nodeptr list, int k){
    Nodeptr p, q = NULL;
    int i;
    if (list != NULL && k > 0) {
        p = list;
        for (i = 1; i < k; i++) { // 循环结束时, p指向链表的第k个结点
            p = p->link;
            if (p == NULL) {
                printf("链表中不存在倒数第k个结点!");
                return NULL;
            }
        }
        // p指向链表最后那个结点, q指向倒数第k个结点
        for (q = list; p->link != NULL; p = p->link, q = q->link)
            ;
    }
    return q; // 给出链表倒数第k个结点(q指向的那个结点)的地址
}
```



综合应用D02-04：词频统计 – 链表

- ◆问题：编写程序统计一个文件中每个单词的出现次数（词频统计），并按字典序输出每个单词及出现次数。
- ◆算法分析：本问题算法很简单，基本上只有查找和插入操作





D02-04: 词频统计 – 链表

由于本问题有如下特点：

1. 问题规模不知（即需要统计的单词数量未知）
2. 单词表需要频繁的执行插入操作

因此，采用顺序表（数组）来构造单词表面临如下问题：

1. 单词表长度太小，容易满，太大，空间浪费
2. 插入操作效率低（经常需要移动大量数据）

而链表具有动态申请结点（空间利用率高），插入和删除结点操作不需要移动结点，插入和删除算法效率高！但单词查找效率低（不能用折半等高效查找算法）



代码实现

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#define MAXWORD 32
struct node {
    char word[MAXWORD];
    int count;
    struct node *link;
} ; //单词表结构
//单词表头指针, 使用全局变量
struct node *Wordlist = NULL;
int getWord(FILE *bfp, char *w);
int searchWord(char *w);
int insertWord( struct node *p, char *w);
```

```
int main()
{
    char filename[32], word[MAXWORD];
    FILE *bfp;
    struct node *p;

    scanf("%s", filename);
    if((bfp = fopen(filename, "r")) == NULL){
        fprintf(stderr, "%s can't open!\n", filename);
        return -1;
    }
    //从文件中读入一个单词
    while( getWord(bfp, word) != EOF)
        //在单词表中查找插入单词
        if(searchWord(word) == -1) {
            fprintf(stderr, "Memory is full!\n");
            return -1;
        }
    //遍历输出单词表
    for(p=Wordlist; p != NULL; p=p->link)
        printf("%s %d\n", p->word, p->count);
    return 0;
}
```



特点分析

◆采用链表方式构造单词表具有如下特点：

✓优点

- 由于采用动态申请结点，能够适应不同规模的问题，空间利用率高
- 算法简单，插入操作效率高

✓不足

- 由于采用顺序查找，单词查找效率低

还有更好的单词表的构造及查询方法吗？



综合应用D02-05：多项式相加（链表实现）

- ◆【问题描述】编写一个程序实现任意（最高指数为任意正整数）两个一元多项式相加。
- ◆【输入形式】从标准输入中读入两行以空格分隔的整数，每一行代表一个多项式，且该多项式中各项的系数均为0或正整数。对于多项式 $a^n x^n + a^{n-1} x^{n-1} + \dots + a^1 x^1 + a^0 x^0$ 的输入方法如下：
 $a^n \ n \ a^{n-1} \ n-1 \ \dots \ a^1 \ 1 \ a^0 \ 0$ ，即相邻两个整数分别表示表达式中一项的系数和指数。在输入中只出现系数不为0的项。
- ◆【输出形式】将运算结果输出到屏幕。将系数不为0的项按指数从高到低的顺序输出，每次输出其系数和指数，均以空格分隔。最后要求换行。
- ◆【样例输入】
54 8 2 6 7 3 25 1 78 0
43 7 4 2 8 1
- ◆【样例输出】
54 8 43 7 2 6 7 3 4 2 33 1 78 0
- ◆【样例说明】输入的两行分别代表如下表达式：
 $54x^8 + 2x^6 + 7x^3 + 25x + 78$
 $43x^7 + 4x^2 + 8x$
其和为： $54x^8 + 43x^7 + 2x^6 + 7x^3 + 4x^2 + 33x + 78$

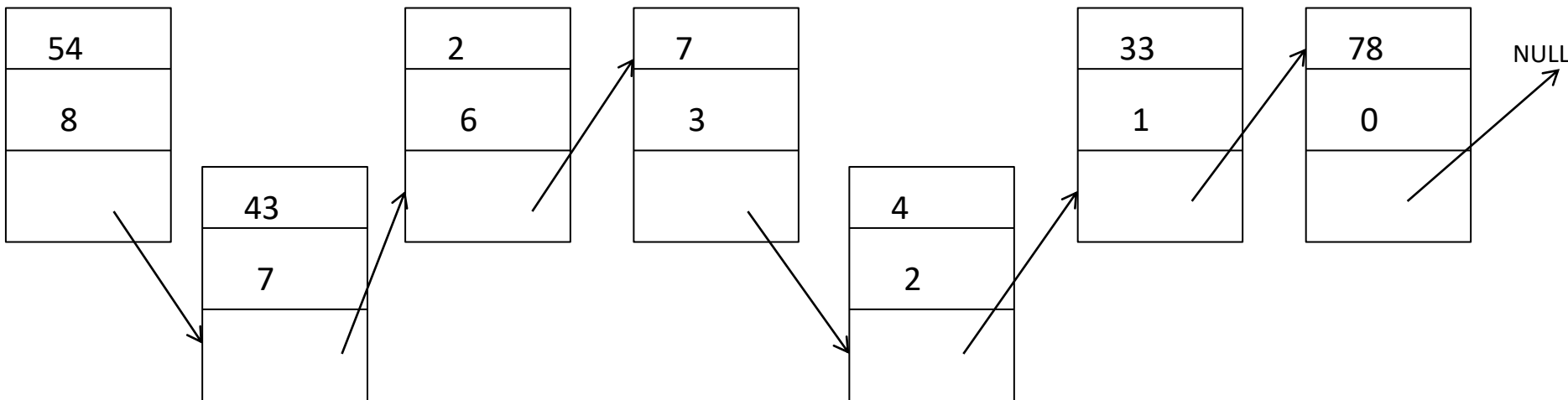
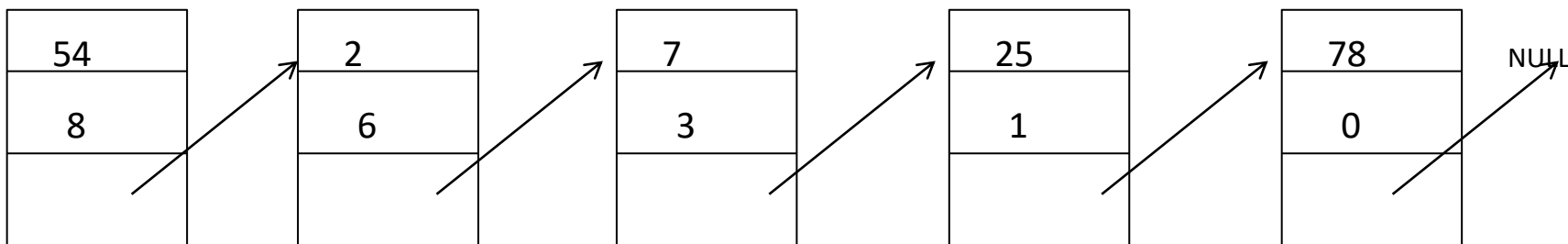


算法设计

◆ 首先读入第一个多项式，并将其生成一个链表

$$54x^8 + 2x^6 + 7x^3 + 25x + 78$$
$$43x^7 + 4x^2 + 8x$$

◆ 然后依次将第二个多项式每一项插入（或合并）第一个多项式中





◆将第二个多项式一个结点加到第一个多项式中有下面几种情况

- ✓第一个多项式中存在相同指数的结点：系数直接相加
- ✓第一个多项式中不存在相同指数的结点：插入到某个结点前
- ✓第一个多项式中不存在相同指数的结点：插入到头指针前（才有头结点，则不需考虑此类问题），或尾结点后



算法实现：相关类型定义

```
#include <stdio.h>
#include <stdlib.h>
typedef struct _ElemType
{
    int coe; // 系数
    int pow; // 指数
} ElemType;
typedef struct _Node // 多项式结点结构体
{
    ElemType data;
    struct _Node *link;
} Node, *Nodeptr;
// 创建一个新结点
Nodeptr createNode(ElemType item);
// 插入到有序链表中，按照指数降序插入，如果指数相同，则系数相加
void insertSortNode(Nodeptr list, ElemType item);
// 销毁链表
Nodeptr destroyList(Nodeptr list);
```



算法实现：主函数

```
int main(){
    ElemType item;  char ch;
    Nodeptr list;
    item.coe = 0;   item.pow = 0;
    list = createNode(item); // 创建头结点
    do { // 创建一个有序链表存放第一个多项式
        scanf("%d%d%c", &item.coe, &item.pow, &ch); // 读入第一行系数和指数
        insertSortNode(list, item); // 插入到有序链表中
    } while (ch != '\n');
    do { // 将第二个多项式的每个项插入到第一个多项式链表中
        scanf("%d%d%c", &item.coe, &item.pow, &ch); // 读入第二行系数和指数
        insertSortNode(list, item); // 插入到有序链表中，重复项合并
    } while (ch != '\n');
    for (p = list; p != NULL; p = p->link) { // 输出结果
        if (p->data.coe != 0) // 系数不为0的时候输出
            printf("%d  %d  ", p->data.coe, p->data.pow);
    }
    list = destroyList(list); // 销毁链表
    return 0;
}
```




算法实现：插入有序结点

// 插入到有序链表中，按照指数降序插入，如果指数相同，则系数相加

```
void insertSortNode(Nodeptr list, ElemType item){
```

```
    Nodeptr p, q, r; // q为插入位置前一个结点、p为插入位置后一个结点，r为新建结点
```

```
    // 找到插入位置
```

```
    for (q = list, p = list->link; p != NULL && p->data.pow > item.pow; q = p, p = p->link)
```

```
    ;
```

```
    if (p == NULL) { // 如果插入到最后
```

```
        r = createNode(item);
```

```
        q->link = r;
```

```
    }
```

```
    else {
```

```
        if (p->data.pow == item.pow) // 如果指数相同
```

```
            p->data.coe += item.coe;
```

```
        else { // q后面，p前面插入
```

```
            r = createNode(item);
```

```
            r->link = p;
```

```
            q->link = r;
```

```
        }
```

```
    }
```

```
}
```

// 创建一个新结点，返回结点地址

```
Nodeptr createNode(ElemType item)
```

```
{
```

```
    Nodeptr p;
```

```
    // 分配结点空间
```

```
    p = (Nodeptr)malloc(sizeof(Node));
```

```
    p->data = item;
```

```
    p->link = NULL; // 指针域设置为NULL
```

```
    return p;
```

```
}
```



算法实现：销毁链表

```
// 销毁链表所有结点（含头结点）
Nodeptr destroyList(Nodeptr list)
{
    Nodeptr p = list;
    while (list != NULL) // 链表不为空
    {
        p = list->link; // p移到下一个结点
        free(list);      // 释放第一个结点
        list = p;        // 指向下一个结点
    }
    return NULL;
}
```



链式存储结构的特点

◆优点

- ✓(1) 存储空间动态分配，可以根据实际需要使用
- ✓(2) 不需要地址连续的存储空间（不需要大块连续空间）
- ✓(3) 插入/删除操作只须通过修改指针实现，不必移动数据元素，算法效率高：
时间复杂度 $O(1)$

◆缺点

- ✓(1) 每个链结点需要设置指针域（占用存储空间小），指针操作比较复杂
- ✓(2) 作为一种非连续存储结构，查找、定位等操作要通过顺序遍历链表实现，
算法效率较低：时间复杂度 $O(n)$



顺序结构和链式结构的比较

存储分配方式

- 顺序存储用一段连续的存储单元依次存储线性表的数据元素
- 链表采用链式储存结构, 用一组不连续的存储单元存放线性表的元素

时间性能

- 查找
 - 顺序存储: 无序 $O(n)$, 有序 $O(\log_2 n)$
 - 链表 $O(n)$
- 插入和删除
 - 顺序存储需要平均移动表长一半的元素, 时间为 $O(n)$
 - 链表在给出结点位置后, 插入和删除时间仅为 $O(1)$

空间性能

- 顺序存储需要事先分配存储空间, 分大了浪费, 分小了易发生溢出
- 链表不需要事先分配存储空间, 需要时分配结点, 元素个数不受限制

结论:

1. 若线性表需要频繁查找 (通讯录), 较少进行插入和删除操作时, 宜采用顺序存储结构。若需要频繁插入和删除时 (如单词表), 宜采用链表结构
2. 当线性表中的元素个数变化较大或者根本不知道有多大时 (如单词表), 最好用链表结构。而如果事先知道线性的大致长度, 用顺序结构效率会高些



提纲：线性表

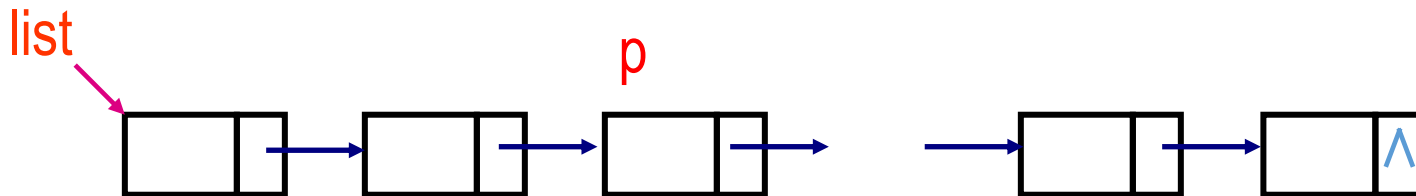
- ◆ 2.1 线性表的基本概念
- ◆ 2.2 线性表的顺序存储
- ◆ 2.3 线性表的链式存储：线性链表
- ◆ 2.4 循环链表
 - ✓ 2.4.1 基本原理和操作
 - ✓ 2.4.2 综合应用
- ◆ 2.5 双向链表



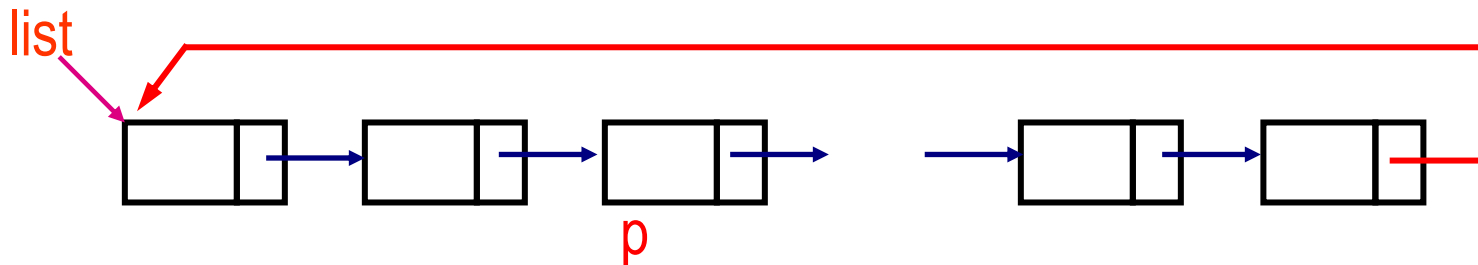
2.4 循环链表

◆ 循环链表是指链表中最后那个链结点的指针域存放指向链表最前面那个结点的指针，整个链表形成一个环

线性链表



循环链表





求链表的长度：普通链表和循环链表

//单链表：返回list链表的长度（结点个数）

```
int getLength(Nodeptr list)
{
    Nodeptr p; //临时指针，用于遍历链表
    int n = 0; //初始化链表长度为0
    //遍历链表，注意3个表达式的写法
    for (p = list; p != NULL; p = p->link)
        n++; //结点个数+1
    return n;
}
```

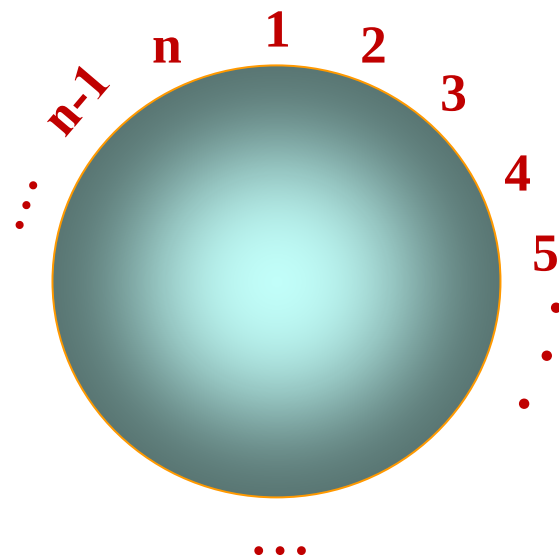
// 循环链表：返回list链表的长度（结点个数）

```
int getLength(Nodeptr list)
{
    Nodeptr p ; // 临时指针，用于遍历链表
    int n = 0; // 初始化链表长度为0
    // 遍历链表，注意3个表达式的写法
    for (p = list; p != NULL || p!=list; p = p->link)
        n++; // 结点个数+1
    return n;
}
```



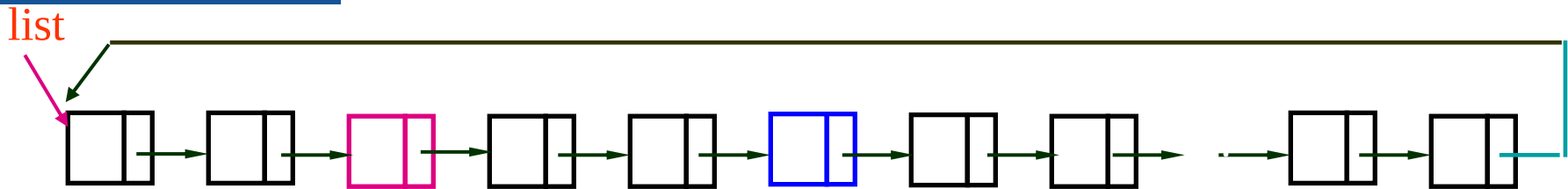
应用D02-06: 约瑟夫 (Josephu) 问题 (圆桌问题)

已知 n 个人(不妨分别以编号 $1, 2, 3, \dots, n$ 代表) 围坐在一张圆桌周围, 编号为 k 的人从1开始报数, 数到 m 的那个人出列, 他的下一个人又从1开始继续报数, 数到 m 的那个人出列, $\dots$, 依此重复下去, 直到圆桌周围的人全部出列





算法设计：使用不带头结点的循环链表



n: 链表中链结点的个数 ($n > 0$)

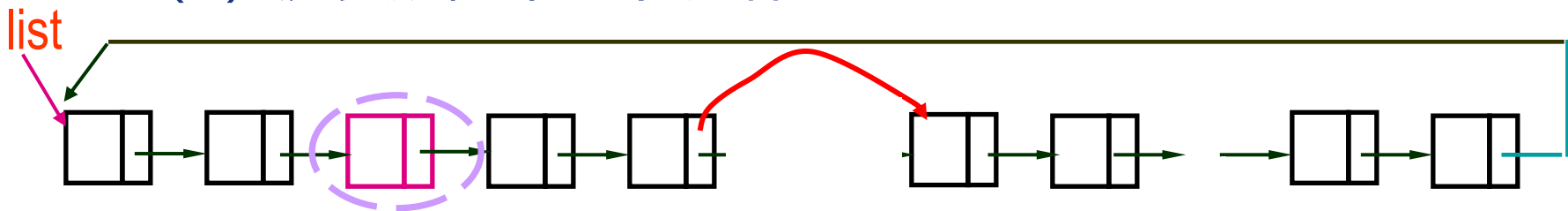
k: 第一个出发点

m: 报数

需要完成的工作:

- (1) 根据n值, 建立循环链表
- (2) 找到第一个出发点
- (3) 反复删除第m个链结点

若假设 $k=3$, $m=4$



`p=p->link;`

k-1次 m-1次



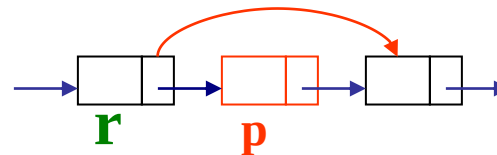
代码实现

// 约瑟夫问题

```
void josephu(int n, int k, int m) {
    Nodeptr list, p, r;
    int i;
    list = NULL;
    for (i = 0; i < n; i++) { // 创建链表
        r = (Nodeptr)malloc(sizeof(Node));
        r->data = i;
        if (list == NULL)
            list = p = r;
        else {
            p->link = r;
            p = p->link;
        }
    }
    p->link = list; // 建立循环链表
    // 找到第一个点
    for (p = list, i = 0; i < k - 1;
        i++, r = p, p = p->link) ;
```

// 循环删除第m个结点

```
while (p->link != p)
{
    for (i = 0; i < m - 1; i++)
    { // 找到第m个点
        r = p;
        p = p->link;
    }
    r->link = p->link;
    printf("%3d", p->data);
    free(p);
    p = r->link;
}
printf("%3d", p->data);
}
```





综合应用D02-07：显示文件最后n行

◆问题：命令tail用来显示一个文件的最后n行。其格式为：

`tail [-n] filename`

✓其中：

➤-n：n表示需要显示的行数，省略时n的值为10

➤filename：给定文件名

➤如：命令tail -20 example.txt 表示显示文件example.txt的最后20行

✓说明：该程序应具有一定的错误处理能力，如能处理非法命令参数和非法文件名

✓示例：

➤若从命令行输入:tail -20 test.txt，以显示文件test.txt的最后20行

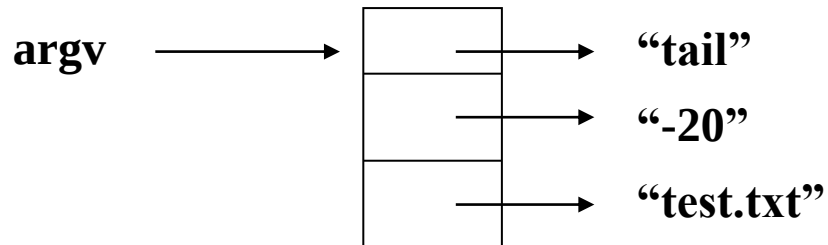


问题分析

◆如何得到需要显示的行数和文件名？

✓使用命令行参数

- `int main(int argc, char *argv[])`
- 行数: `sscanf(argv[1]+1, "%d", &n)`
- 文件名: `filename = argv[2]`



◆如何得到最后n行？

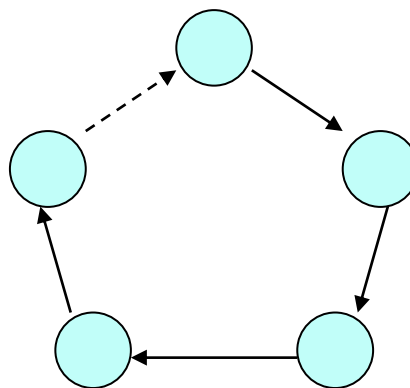
- ✓ 文本文件每行的字符数是不确定的，只能从前往后顺序读写
- ✓ 同时，一次读写完成后，并不能退回到指定的行重新读写



算法设计

◆方法一：使用 n 个结点的循环链表。链表中始终存放最近读入的 n 行

- ✓首先创建一个空的循环链表；
- ✓然后再依次读入文件的每一行挂在链表上，最后链表上即为最后 n 行





代码实现

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#define DEFLINES 10 //默认显示10行
#define MAXLEN 81 //单行文本不超过80
typedef struct _Node
{
    char *line; // 采用指针，新建的时候分配空间
    struct _Node *link;
} Node, *Nodeptr;
// 创建链表的一个结点
Nodeptr createNode(const char *line);
// 销毁循环链表
Nodeptr destroyList(Nodeptr list);
```



代码实现：主函数

```
int main(int argc, char *argv[]) {
    char curline[MAXLEN], *filename;
    int n = DEFLINES, i;
    Nodeptr list, p;
    FILE *fp;
    if (argc == 3 && argv[1][0] == '-') {
        sscanf(argv[1] + 1, "%d", &n);
        filename = argv[2];
    }
    else if (argc == 2)
        filename = argv[1];
    else {
        printf("Usage: tail [-n] filename\n");
        return 1;
    }
    if ((fp = fopen(filename, "r")) == NULL)
    {
        printf("Can't open file: %s !\n", filename);
        return -1;
    }
}
```



代码实现：主函数

```
list = p = createNode(NULL);
for (i = 1; i < n; i++) {
    p->link = createNode(NULL);
    p = p->link;
}
p->link = list;
p = list;
while (fgets(curline, MAXLEN, fp) != NULL) {
    // 链表已经满了，需要释放掉原来的内容，保持新的内容
    if (p->line != NULL) free(p->line);
    p->line = (char *)malloc(strlen(curline) + 1);
    strcpy(p->line, curline);
    p = p->link;
}
for (i = 0; i < n; i++) { // 输出循环链表中的内容
    if (p->line != NULL) printf("%s", p->line);
    p = p->link;
}
destroyList(list);
fclose(fp);
return 0;
}
```




代码实现：辅助函数

// 创建链表的一个结点

```
Nodeptr createNode(const char *line){
    Nodeptr p = (Nodeptr)malloc(sizeof(Node));
    if (line != NULL) {           // 分配内容空间
        p->line = (char *)malloc(strlen(line) + 1);
        strcpy(p->line, line); // 拷贝字符串
    }
    else p->line = NULL;
    p->link = NULL;
    return p;
}
```

```
Nodeptr destroyList(Nodeptr list){// 销毁循环链表
    Nodeptr p = list;
    if (list == NULL) return NULL;
    while (list != list->link)
    {
        // 循环链表剩余超过一个结点
        p = list->link; // p移到下一个结点
        list->link = p->link;
        free(p->line); // 释放结点的字符串内容空间
        free(p); // 释放结点
    }
    free(list->line); free(list); // 释放最后一个结点
    return NULL;
}
```



其他方法

◆方法：两次扫描文件

- ✓第一遍扫描文件，用于统计文件的总行数 N ；
- ✓第二遍扫描文件时，首先跳过前面 $N-n$ 行，只读取最后 n 行。
- ✓如何开始第二遍扫描？
 - `fseek(fp, 0, SEEK_SET);` -- 将文件读写位置移至文件头
 - 或（关闭后）再次打开同一个文件

请同学们考虑是否还有其它方法？甚至更好的方法？

如：我们可设计这样两个函数：

`fbgetc(fp);` //从文件尾开始，每次倒读一个字符

`fbgets(buf, size, fp);` //从文件尾开始每次倒读一行



综合应用D02-08：文件加密（环）

◆【问题描述】有一种文本文件加密方法，其方法如下：

- ✓ 1. 密钥由所有ASCII码可见字符（ASCII码编码值32-126为可见字符）组成，长度不超过32个字符；
- ✓ 2. 先将密钥中的重复字符去掉，即：只保留最先出现的字符，其后出现的相同字符都去掉；
- ✓ 3. 将不含重复字符的密钥和其它不在密钥中的可见字符（按字符升序）连成一个由可见字符组成的环，密钥在前，密钥的头字符为环的起始位置；
- ✓ 4. 设原密钥的第一个字符（即环的起始位置）作为环的开始位置标识，先从环中删除第一个字符（位置标识则移至下一个字符），再沿着环从下一个字符开始顺时针以第一个字符的ASCII码值移动该位置标识至某个字符，则该字符成为第一个字符的密文字符；然后从环中删除该字符，再从下一个字符开始顺时针以该字符的ASCII码值移动位置标识至某个字符，找到该字符的密文字符；依次按照同样方法找到其它字符的密文字符。当环中只剩一个字符时，则该剩下的最后一个字符的密文为原密钥的第一个字符

◆【输入形式】

- ✓ 密钥是从标准输入读取的一行字符串，可以包含任意ASCII码可见字符（ASCII码编码值32-126为可见字符），长度不超过32个字符

◆【输出形式】

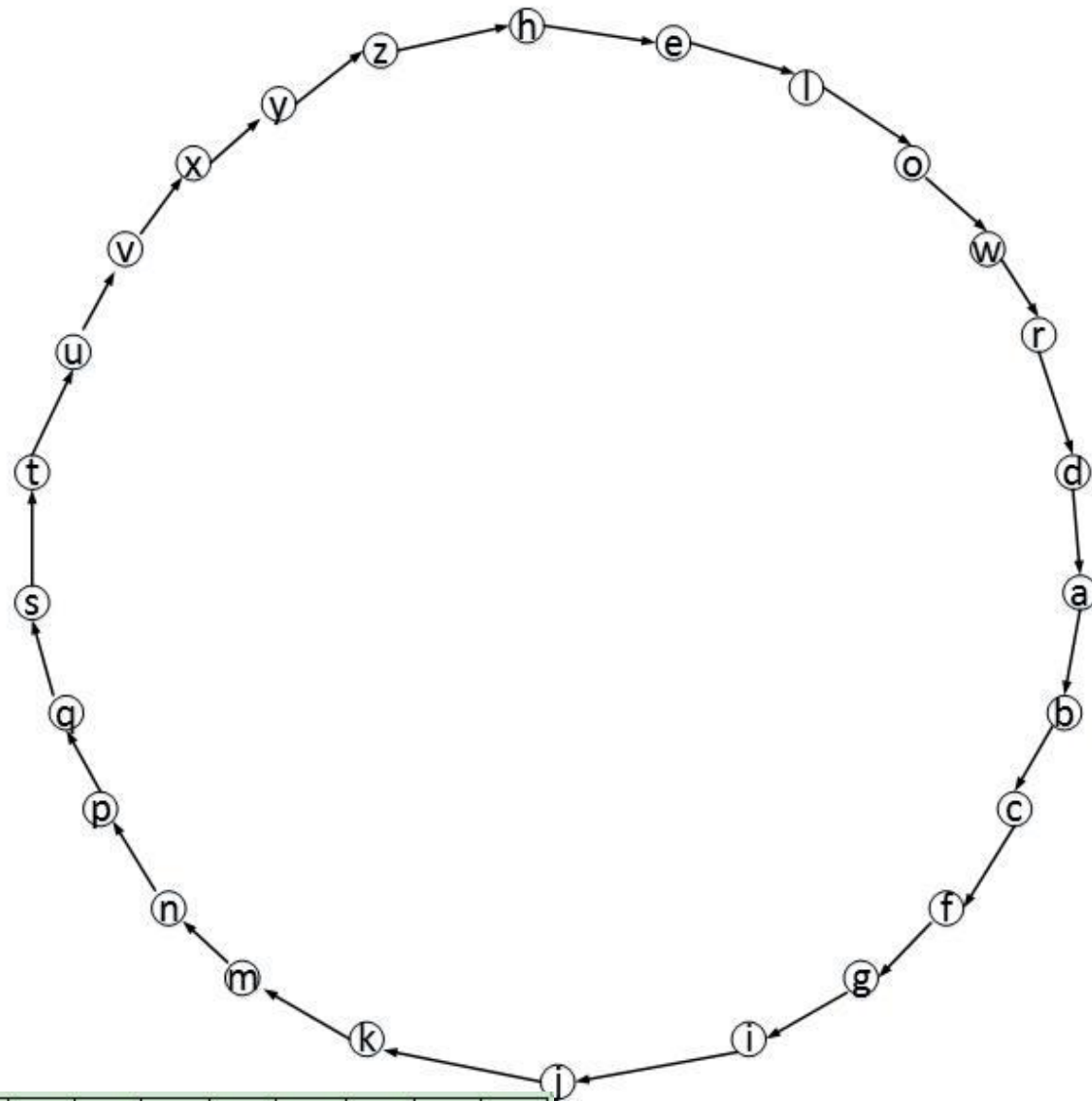
- ✓ 加密后生成的密文文件为当前目录下的in_crpyt.txt



环加密

◆ 示例

- ✓ 如果密钥为：helloworld，将密钥中重复字母按照升序添加在密钥后，即形成字符串：he
- ✓ 明码的第一个字母为h，h也是环的起始位除，再从下一个字母e开始顺时针沿着环按移动第1次，共移动位置标识104次) 至字



a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z
q	g	k	c	d	r	o	w	x	u	t	s	f	i	m	b	z	a	y	v	n	j	l	h	p	e



算法设计要点

◆ 1. 去除重复字符

- ✓ 一个字符一个字符读入，如果已经存在，则不保存

◆ 2. 建立循环链表

- ✓ 逐一申请新结点，添加到链表末尾
- ✓ 最后一个结点链接到第一个结点

◆ 3. 删除链结点

- ✓ 需要保留记录前一个节点信息
- ✓ 通过前一个结点直接链接到后一个结点，删除当前结点

◆ 4. 明文和密文对应关系

- ✓ 采用数组，明文字符的ASCII作为数组下标，内容为密文



总体设计

1. 定义结构体对象，存储字符串环；
2. 读入密钥，并去重；（函数：removeDup）
3. 根据密钥生成字符串表，补充不在密钥中的字符；
4. 根据字符串表，构建字符串环（循环链表）；（函数：buildCircle）
5. 根据字符串环，依次删除元素，生成密钥对；（函数：getPasswordTab）
6. 利用密钥对，加密原始文件；（函数：encodeFile）

```
//定义结构体，存储数据
typedef struct _Node{
    char ch;
    struct _Node *link;
}Node, *PNode;
```

```
void removeDup(char *key); //字符串去重
PNode buildCircle(char key[], char allchar[]); //根据密钥生成字符串
void getPasswordTab(PNode list, char passtab[]); //计算密钥对
void encodeFile(char tab[], char *src, char *dest); //加密文件
```



参考代码

```
int main(){
    char key[40], passtab[95];
    int i;
    PNode list = NULL;
    gets(key);
    removeDup(key); //去除重复字符
    for (i = 0; i < 95; i++)
        passtab[i] = i + ' '; //生成全部字符串表
    for (i = 0; key[i] != '\0'; i++)
        passtab[key[i] - ' '] = '\0'; //删除密钥中的字符
    list = buildCircle(key, passtab); //根据密钥和剩余的字符串构造字符串环

    getPasswordTab(list, passtab); //生成密钥对

    encodeFile(passtab, "in.txt", "in_crpyt.txt"); //利用密钥对加密文件

    return 0;
}
```

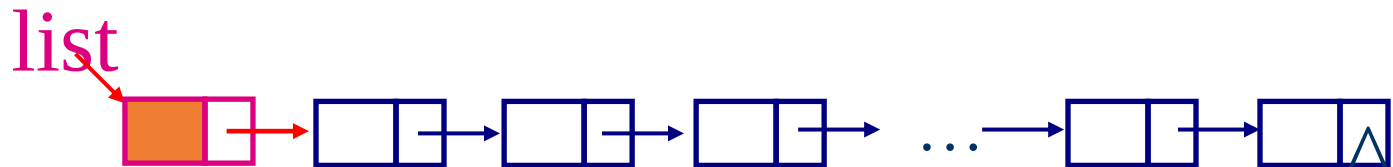


小结：线性表的链式存储

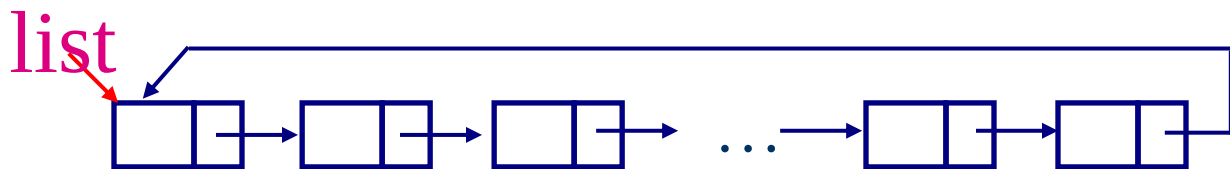
线性链表(单链表)



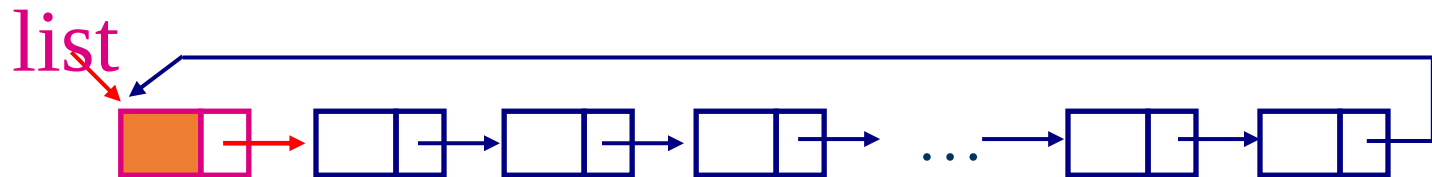
带头结点的线性链表



循环链表



带头结点的循环链表





提纲：线性表

- ◆ 2.1 线性表的基本概念
- ◆ 2.2 线性表的顺序存储
- ◆ 2.3 线性表的链式存储：线性链表
- ◆ 2.4 循环链表
- ◆ 2.5 双向链表



2.5 双向链表

- ◆ 双向链表是指链表的每一个结点中除了数据域以外设置**两个指针域**，其中之一指向结点的直接**后继结点**，另外一个指向结点的直接**前驱结点**



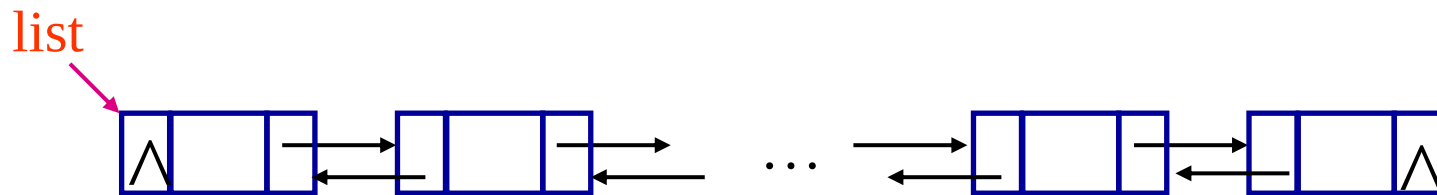
- ◆ 其中:

- ✓ data 为数据域
- ✓ rlink, llink 分别为指向该结点的直接后继结点与直接前驱结点的指针域（右指针和左指针）

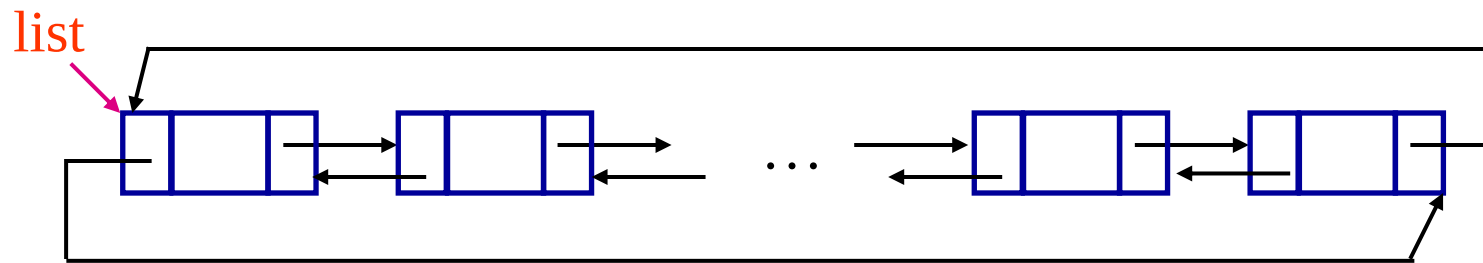
```
typedef struct _DNode {  
    ElemType data;  
    struct _DNode *rlink, *llink;  
} DNode, DNodeptr;
```



双向链表的几种形式



双向链表

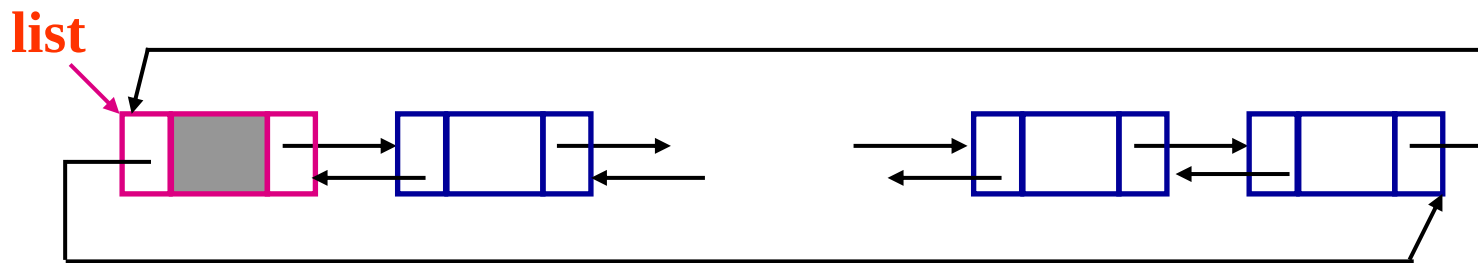


双向循环链表



双向链表的插入

- ◆ 在非空双向循环链表中某个数据域的内容为 x 的链结点右边插入一个数据信息为 $item$ 的新结点



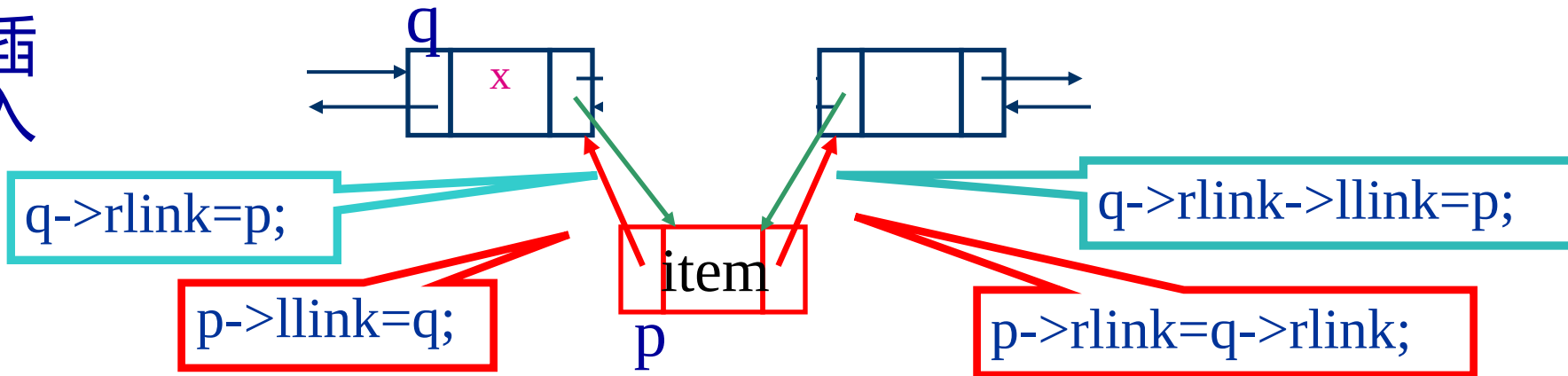
需要做的工作:

1. 找到满足条件的结点;
2. 若找到, 构造一个新的链结点;
3. 将新结点插到满足条件的结点后面。

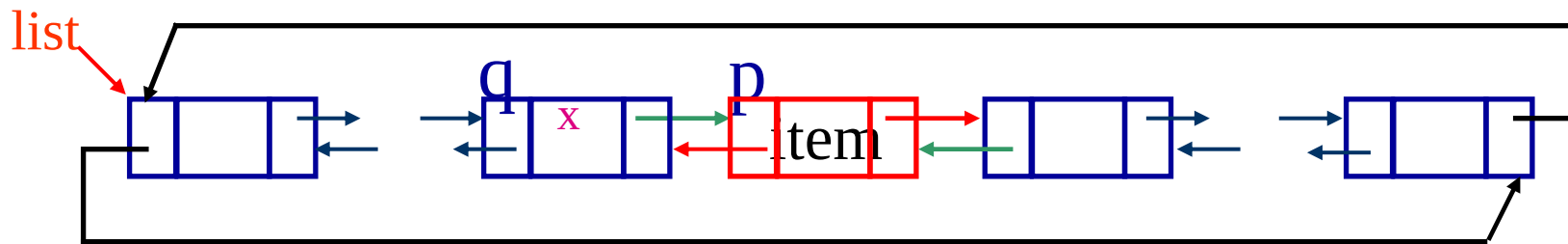


双向链表的插入

插入



插入后



注意：在头(第一个)指针前插入一个结点时，步骤如下：

```
p->rlink = list; p->llink = list->llink;
```

```
list->llink->rlink = p; list->llink = p; list = p;
```



算法实现

//在双向循环链表中值为x的结点后面插入item

```
int insertDNode(DNodeptr list, ElemType x, ElemType item)
```

```
{
```

```
    DNodeptr p, q; //p为新结点, q指向值为x的结点
```

```
    //寻找满足条件的结点
```

```
    for (q = list; q && q->data != x && q->rlink != list; q = q->rlink)
```

```
    ;
```

```
    if (!q || q->data != x) //没有找到满足条件的结点
```

```
        return -1;
```

```
    //创建新结点
```

```
    p = (DNodeptr)malloc(sizeof(DNode));
```

```
    p->data = item;
```

```
    //在q的右边（后面）插入
```

```
    p->llink = q;           //p的左指针指向q（前一个结点）
```

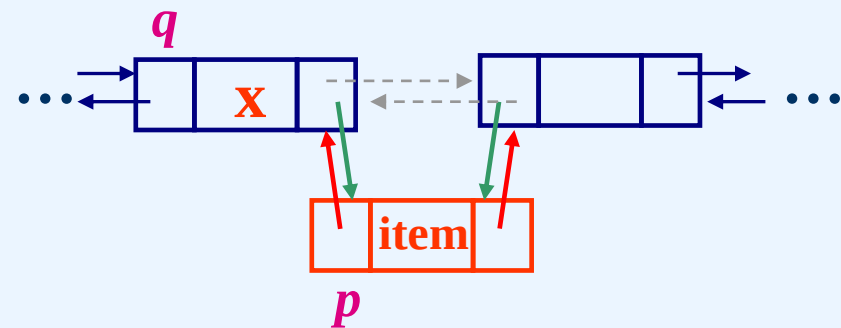
```
    p->rlink = q->rlink; //p的右指针指向q的下一个结点
```

```
    q->rlink->llink = p; //q的后一个结点的左指针指向p
```

```
    q->rlink = p;          //q的右指针指向p
```

```
    return 1;
```

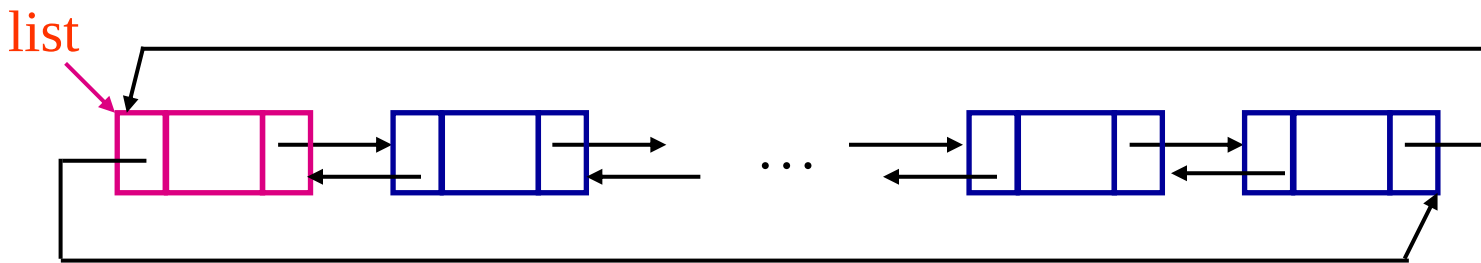
```
}
```





双向链表的删除

删除非空双向循环链表中数据域的内容为 x 的链结点



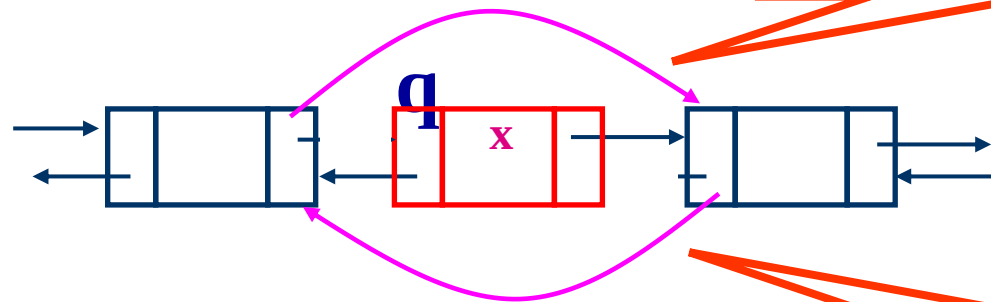
需要做的工作:

1. 找到满足条件的结点;
2. 若找到, 删除(并释放)满足条件的结点。



双向链表的删除

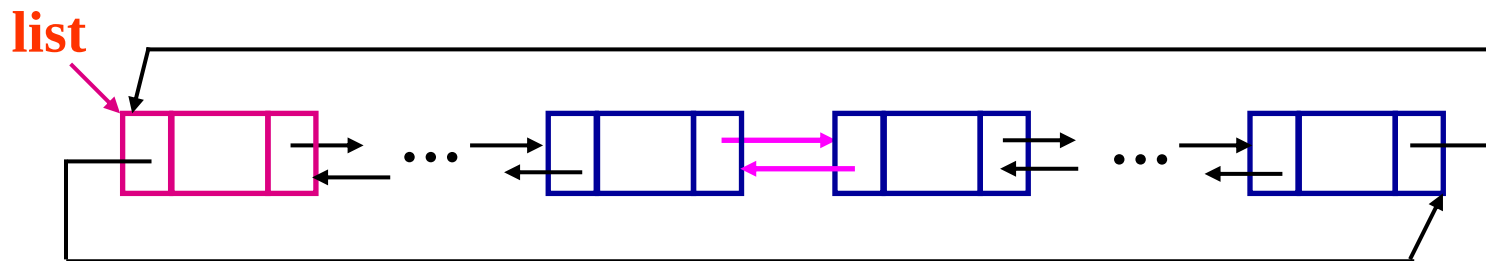
删除



$q \rightarrow \text{llink} \rightarrow \text{rlink} = q \rightarrow \text{rlink};$

$q \rightarrow \text{rlink} \rightarrow \text{llink} = q \rightarrow \text{llink};$

删除后



注意：删除头(第一个)结点时，步骤如下：

```
list->rlink->llink = list->link;  
list->llink->rlink = list->rlink;  
q = list; list = list->rlink; free(q);
```

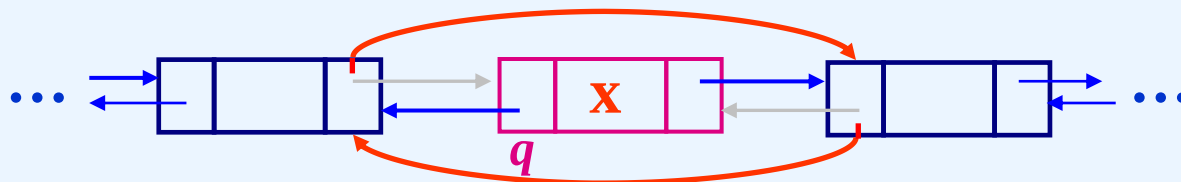



算法实现

非空双向循环链表的删除操作，若按不带头结点处理，需要考虑如下三种情况：

- 1) 删除的是第一个结点
- 2) 删除的是只有唯一一个结点的链表
- 3) 正常的删除

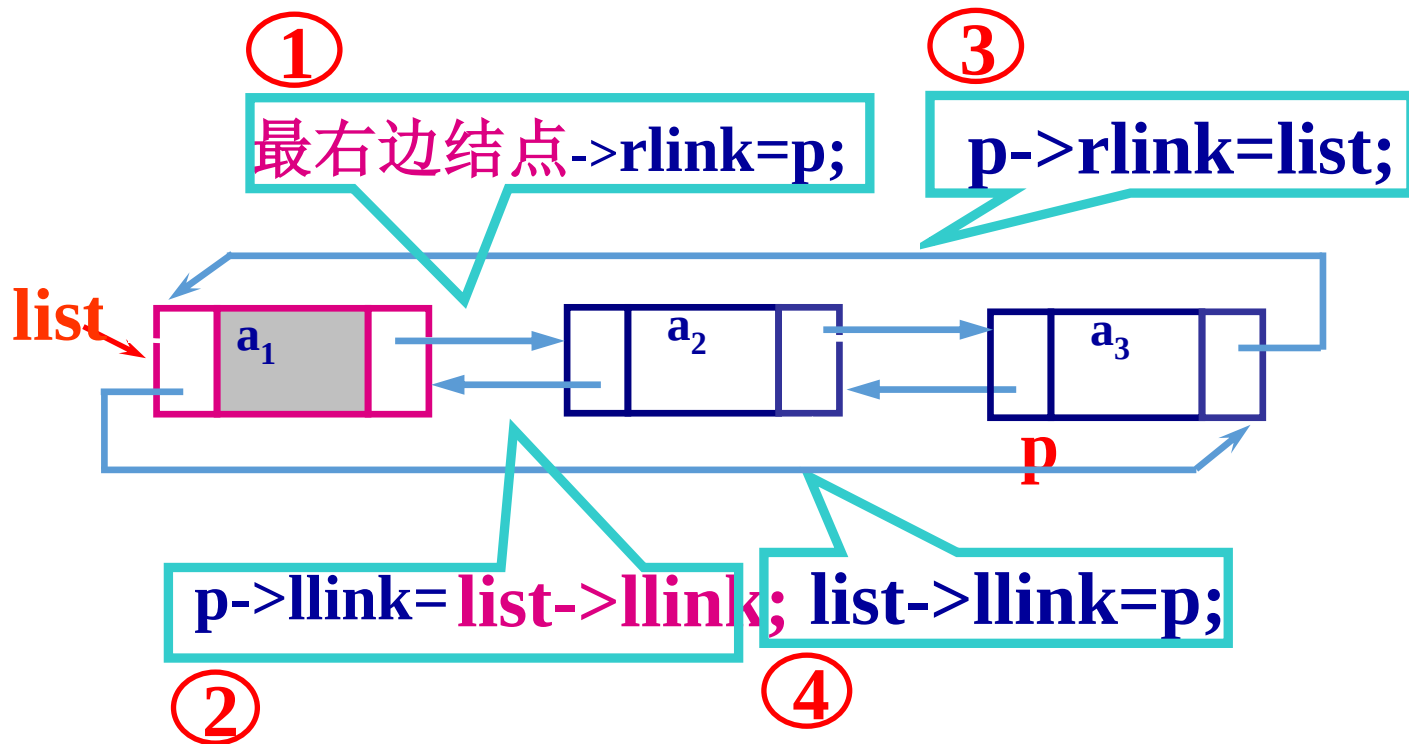
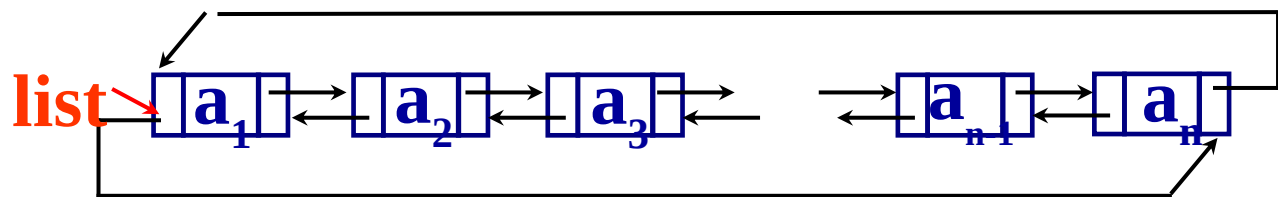
```
int deleteDNode(DNodeptr list, ElemType x)
{
    DNodeptr q;
    // q初始指向第一个结点，头指针
    for (q = list; q && q->data != x && q->rlink != list; q = q->rlink)
        ;
    // 没有找到满足条件的结点
    if (!q || q->data != x)
        return -1;
    q->llink->rlink = q->rlink;
    q->rlink->llink = q->llink;
    free(q); //释放被删除的结点的存储空间
    return 1;
}
```





构造一个双向循环链表

$(a_1, a_2, a_3, \dots, a_{n-1}, a_n)$





算法实现

//创建n个结点的双向循环链表($n>0$)

DNodeptr createDLink(int n)

```
{
    int i;
    DNodeptr list, p;
    //建第一个结点
    list = (DNodeptr)malloc(sizeof(DNode));
    READ(list->data); //读取第一个数据存入结点
    list->llink = list;
    list->rlink = list;
    for (i = 1; i < n; i++)
    {
        p = (DNodeptr)malloc(sizeof(DNode));
        READ(p->data); // 读取数据存入结点
        insertNode(list, p); //插入到循环链表
    }
    return list;
}
```

//双向循环链表的后面插入结点p

```
void insertNode(DNodeptr list, DNodeptr p)
{
    list->llink->rlink = p;
    p->llink = list->llink;
    p->rlink = list;
    list->llink = p;
}
```



双向链表特点小结

- ◆双向链表由于多了一个前驱结点的指针，使得结点的插入和删除时需要做更多的操作，需要小心
- ◆双向链表需要保存前驱和后续结点的指针，要比单向链表多占用一些空间
- ◆双向链表由于很好的对称性，使得对某个结点的前后结点访问带来了方便，简化了算法，可以提高算法的时间性能，以空间换时间



线性链表使用的注意事项

链表使用的注意事项:

- ① 应确保链表结点指针指向一个合法空间（通常由malloc申请而得），否则结点操作时会出现内存错误（**memory access violation**），如：

```
struct Node *p;  
p->link = q;
```
- ② 单向链表的最后一个结点的p->link指针一定要为NULL,通常申请一个结点p时及时执行**p->link = NULL;**语句是一个好习惯;
- ③ 当链表结点删除后应及时用free(p)释放。不释放不用的结点会造成内存泄漏（**memory leak**），这是工程应中常见问题;
- ④ 不能随意移动链表的头指针，对头指针的移动都应是有意义的（如要在头指针前插入一个结点或要删除链表的第一个结点），否则会造成链表头指针的丢失。



小结

◆线性表的基本概念

- ✓线性关系、线性表和基本操作

◆线性表的顺序存储

- ✓构造原理；查找、插入、删除等算法设计 and 应用

◆线性链表（单链表）

- ✓基本结构定义；查询、插入、删除等算法设计；头结点
- ✓单链表的综合应用

◆循环链表

- ✓循环链表的综合应用

◆双向链表